UNIDAD 1
INICIANDO CON C++

Por Félix Ferreiras
y Zobiesky Ovalle

Elementos del lenguaje: mi primer programa

Objetivos

En esta unidad usted aprenderá a:

- Describir los conceptos claves del lenguaje C++.
- Conocer las variables y los tipos de datos.
- Utilizar los operadores en base a su orden de ejecución.
- Instalar nuestro ambiente de programación (compilador y herramientas).
- Analizar problemas informáticos a partir de un enfoque básico.
- Construir nuestro primer programa C++.
- Conocer las operaciones de entrada y salida en C++.

Contenido

- Elementos del lenguaje.
- Variables y tipos de datos.
- Operadores (aritméticos, de relación, lógicos).
- Ambiente de programación (compilador, herramientas).
- Análisis del problema.
- Mi primer programa en C++.
- Operaciones de entrada y salida.

Contenido

- Elementos del lenguaje.
- Variables y tipos de datos.
- Operadores (aritméticos, de relación, lógicos).
- Ambiente de programación (compilador, herramientas).
- Análisis del problema.
- Mi primer programa en C++.
- Operaciones de entrada y salida.

Elementos del lenguaje

Identificadores, palabras reservadas, literales, tipos de datos, sentencias, bloque de código, comentarios

Identificadores

Caracteres permitidos:

Alfabéticos

Dígitos

Subrayado

- Dentro de un programa podemos tener diversos elementos, tales como: variables, funciones, constantes, estructuras, etc.
- Estos elementos deben ser nombrados con ciertas normas de acuerdo a las reglas del lenguaje de programación utilizado.
- De forma general esto es referido como identificadores

Identificadores

- Los identificadores están sujetos a las siguientes reglas de escritura:
  - El primer carácter debe ser alfabético o un signo de subrayado (no recomendado)
  - Los siguientes caracteres (del segundo en adelante) pueden ser alfabéticos, dígitos o subrayado
  - Los identificadores deben ser descriptivos, sugerentes al propósito de su uso, por ejemplo: un nombre de variable para almacenar la edad promedio de un grupo de personas podría ser *edadPromedio*

Palabras reservadas (key words)

Las palabras reservadas no deben ser utilizadas como identificadores en un programa.

asm	dynamic_cast	new	template
auto	else	operator	this
bool	enum	private	throw
break	extern	protected	true
case	false	public	try
catch	float	register	typedef
char	for	reinterpret_cast	typeid
class	friend	return	union
const	goto	short	unsigned
const_cast	if	signed	using
continue	inline	sizeof	virtual
default	int	static	void
delete	long	static_cast	volatile
do	mutable	struct	wchar_t
double	namespace	switch	while

Identificadores

Ejemplos

- `duracionLlamada`
- `salarioPorDepartamento`
- `peso_especifico`
- `Correo_Electronico`
- `telefonoCasa1`
- `_valorMaximo`

Contra - Ejemplos

- `2valorPromedio`
- `tasa-de-interes`
- `#_de_empleado`
- `em@il_principal`
- `Direccion 1`
- `precioRD$`

Literales

- Un literal es data en sí misma, es lo que se asignaría a una variable o lo que podría participar en una expresión aritmética, lógica o de cadena como un valor fijo
- Los literales pueden ser numéricos, caracter, lógicos, de cadena y especiales

Literales

Los literales pueden ser:

Numéricos
Caracter
De cadena
Lógicos
Especiales

Tipo	Ejemplo 1	Ejemplo 2
Numérico entero	39	-11235813
Numérico de punto flotante	3.14159	-2.7218E15
Carácter	'a'	'='
De cadena (string)	"Hola Mundo!"	"Su promedio es: "
Lógico (booleano)	true	false
Especiales	null	

Comentarios

- Los comentarios ayudan a documentar nuestro programa, esto es ignorado por el compilador y podemos agregar tantas líneas de comentario sean necesarias
- En C++ existen dos tipos de comentarios:
  - Comentario de una línea: //
  - Comentario de múltiples líneas: /* */

Comentarios

- Ejemplo de comentario de una línea

```
// Declaracion de variables
```

- Ejemplo de comentario de múltiples líneas

```
/*
   Funcion que retorna el mayor elemento de un arreglo
   Autor: John Doe
   Creacion: 2013-12-18
*/
```

Sentencia

- Una sentencia en C++ es todo mandato que le indique al compilador realizar una determinada acción. Las sentencias terminan con el caracter ; (punto y coma) el cual separa una sentencia de la siguiente
- Una buen estilo de escribir código fuente consiste en escribir las sentencias unas debajo de otras sin importar lo corta que sean

Bloque de código

- Se refiere a una o más sentencias limitadas por un par de llaves, una llave abierta y una llave cerrada

```
{
    sentencia1;
    sentencia2;
    sentencia3;
 }
```

Contenido

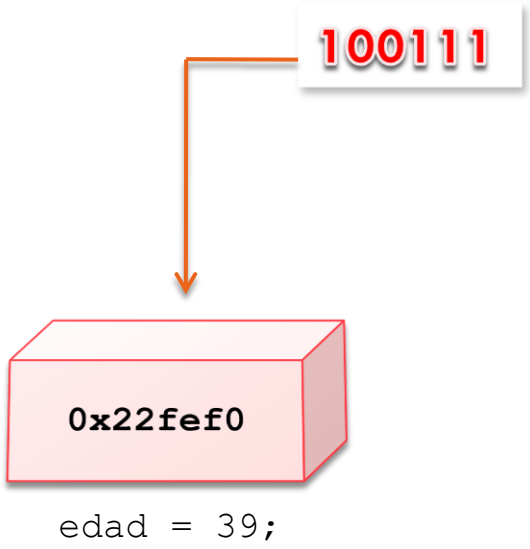
- Elementos del lenguaje.
- Variables y tipos de datos.
- Operadores (aritméticos, de relación, lógicos).
- Ambiente de programación (compilador, herramientas).
- Análisis del problema.
- Mi primer programa en C++.
- Operaciones de entrada y salida.

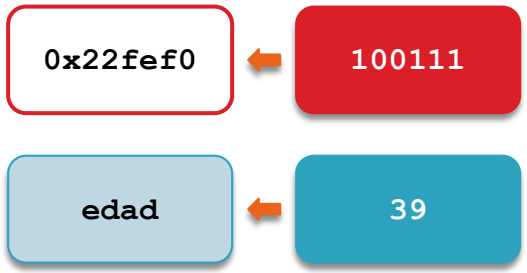
Variables y tipos de datos

Variables, tipos de datos, rango de valores, alcance

Variables

- Una variable es un vehículo que nos permite manipular datos o valores dentro de un programa
- Una variable es un nombre para una posición de memoria en RAM con capacidad de almacenar un dato. La data contenida en la variable puede cambiar durante la ejecución del programa
- Las direcciones de memoria son expresadas en el sistema numérico hexadécimal





Tipos de datos

- C++ cuenta con un conjunto de tipos de datos los cuales no ameritan el uso de ninguna librería pues vienen de forma implícita en el compilador, a estos tipos de datos también le llamamos tipos de datos primitivos
- De igual forma que para los literales, los tipos de datos primitivos (built-in) en C++ son: *char, int, short, long, float, double, bool*

Rango de valores

Tipo	Tamaño	Rango soportado
char	8 bits	De 0 a 255
signed char	8 bits	De -128 a +127
short	16 bits	De -32768 a +32767
unsigned short	16 bits	De 0 a 65535
int	32 bits	De -2,147,483,648 a +2,147,483,647
unsigned int	32 bits	De 0 a 4,294,967,295
long	32 bits	De -2,147,483,648 a +2,147,483,647
unsigned long	32 bits	De 0 a 4,294,967,295
float	32 bits	De 3.4 E-38 a 3.4E+38
double	64 bits	De 1.7 E-308 a 1.7E+308
long double	96 bits	De 3.4 E-4932 a 1.1E+4932
bool	8 bits	true / false

Importante: Los valores reflejados en la tabla podrían variar pues dependen del tipo de plataforma para la cual haya sido compilado nuestro programa así como del compilador mismo que estemos utilizando. Existen ciertas garantías, por ejemplo los datos tipo *long* son por lo menos del mismo tamaño que un dato tipo *int*.

Declaración de variables

- La declaración de variable es la acción de definir un símbolo en nuestro programa fuente para asociar a este una posición de memoria con la cual podamos manipular el espacio reservado. Cuando hablamos de símbolo nos referimos a un nombre de variable el cual debe someterse a las reglas de los identificadores para su escritura

```
tipo nombre;
```

Donde:

tipo es el tipo de dato elegido para la variable

nombre es el nombre elegido para la variable, debe cumplir con la regla de los identificadores

Nota: se coloca punto y coma al final para indicar el final de la sentencia

Inicialización de variables

- Las variables pueden ser inicializadas al momento de su declaración. Esta decisión es muy recomendable sobre todo cuando tenemos la intención de utilizar una variable como un contador o acumulador, generalmente los compiladores C++ no asignan un valor inicial para las variables declaradas no inicializadas por lo que es importante tomar esto en consideración al momento de definir variables. Veamos las dos formas válidas:

```
tipo nombre(valor);
tipo nombre = valor;
```

Donde:

valor es el valor inicial asignado a la variable declarada, este valor puede ser un literal, constante, variable o expresión que al ser evaluada retorne un valor del mismo tipo de la variable o que dicho valor pueda ser convertido de forma implícita al tipo de dato especificado en la sentencia

Nota: Las conversiones implícitas y explícitas de datos serán tratadas mas adelante, a esto es lo que se conoce como Casting

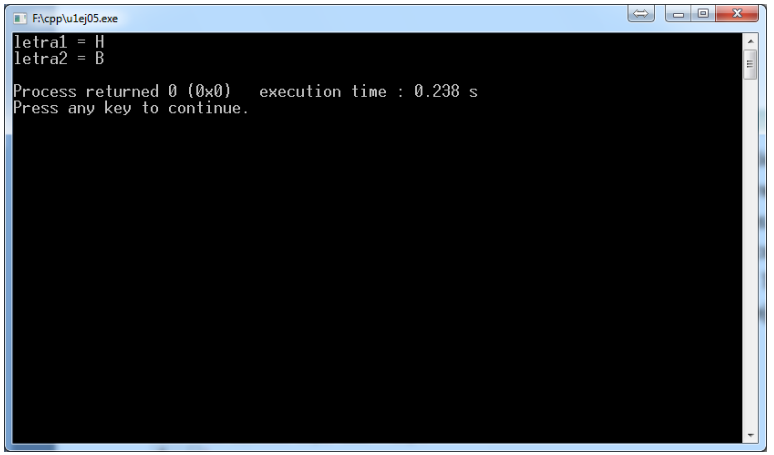
Ejemplos (declaración e inicialización)

```
int contador;
int sumatoriaEdad = 0;
double precio(0.0);
bool encontrado = false;
char seguir;
short pesoInicial(1200);
char opcionPorDefecto = 'N';
float descuento = 0.0;
long factorial = 0;
unsigned int edad;
short bitRate = 320;
```

Tipo de dato char

- En C++ el tipo de dato ***char***, el cual es de 1 byte de capacidad, es utilizado para almacenar valores tipo caracter, aunque en realidad este tipo de dato es de tipo numérico (entero). Si prestamos atención a la tabla de caracteres ASCII, podemos ver que cada caracter tiene asociado un valor entero, y precisamente de 1 byte (0 – 255), es por esto que el tipo de dato ***char*** puede manejar tanto valores enteros como carácter

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       char letra1 = 72;
8       char letra2 = 'B';
9
10      cout << "letra1 = " << letra1 << endl
11           << "letra2 = " << letra2 << endl;
12
13      return 0;
14  }
```



Alcance de las variables

- Dentro de un programa C++, las variables pueden tener distintos tipos de alcance. Para comprender mejor esto, veamos la estructura general de un programa C++:

```
// Directivas de Pre-Procesamiento
#include <iostream>

// Espacio de Nombre Estándar
using namespace std;

// Declaraciones Globales

// Definiciones de Funciones
```

En esta parte definimos tantas librerías necesitemos utilizar

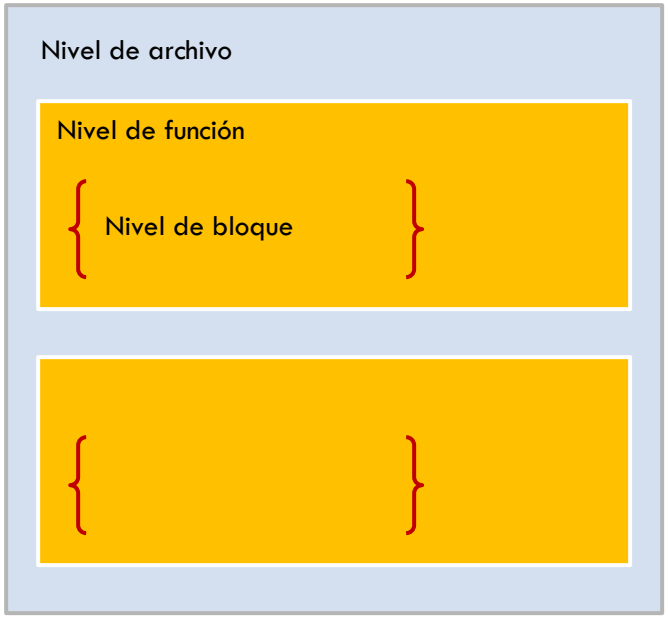
A continuación definimos el nombre de espacio estándar

En esta sección declaramos los elementos globales: variables, estructuras, constantes

Aquí definimos todas las funciones que tendría nuestro programa fuente

Alcance de las variables

- El alcance de las variables podemos definirlo en tres niveles:
  - Nivel de bloque
  - Nivel de función
  - Nivel de archivo



Concepto:

Nivel de archivo: Es el nivel mayor, una variable declarada a nivel de archivo puede ser vista o accedida desde todas las funciones del programa fuente y dentro de cada función desde cualquier bloque de código.

Nivel de función: Una variable declarada a este nivel, puede ser accedida o vista dentro de la función donde fue declarada y dentro de cualquier bloque de código que pueda tener la función. Las variables definidas a nivel de función no pueden ser vistas o accedidas desde otras funciones del mismo programa.

Nivel de bloque: Una variable declarada a este nivel, solamente puede ser accedida o vista dentro del mismo bloque de código donde fue definida.

Más adelante veremos con ejemplos lo que significa todo esto.

Contenido

- Elementos del lenguaje.
- Variables y tipos de datos.
- Operadores (aritméticos, de relación, lógicos).
- Ambiente de programación (compilador, herramientas).
- Análisis del problema.
- Mi primer programa en C++.
- Operaciones de entrada y salida.

Operadores

Aritméticos, de relación, lógicos

Definiciones

- **Operando**: Especifica la data a ser manipulada (un operando puede ser una variable, literal o constante)
- **Operador**: Símbolo que representa la acción específica sobre la data (los principales operadores son los aritméticos, de relación y lógicos)
- **Expresión**: Mezcla de operando(s) y operador(es) que produce un valor al ser evaluada; toda expresión debe tener al menos un operando

Tipos de Operadores

- **Asignación:** Permite asignar el valor de una variable, constante, literal o expresión a otra variable
- **Aritméticos:** Operadores que permiten realizar operaciones aritméticas. Todo resultado producto de una operación con estos operadores da como resultado un valor numérico
- **De relación:** Operadores que permiten realizar operaciones de comparación. Todo resultado producto de una operación con estos operadores da como resultado un valor lógico, un valor de verdad falso o verdadero
- **Lógicos:** Operadores que permiten realizar operaciones lógicas. Todo resultado producto de una operación con estos operadores da como resultado un valor lógico, un valor de verdad falso o verdadero

Operador de asignación

- Copia el valor resultante de evaluar la expresión de la derecha en la variable a su izquierda
- Después de su ejecución la variable almacenará el valor resultante de la expresión

- Forma general:

```
variable = expresión;
```

- Ejemplos:

```
cs = a + 2;
intensidad = 5.6;
```

Operadores aritméticos

Operador	Descripción
+	Positivo unario
-	Negativo unario
+	Suma
-	Resta
*	Multiplicación
/	División
%	Residuo
++	Incremento (Post-Pre)
--	Decremento (Post-Pre)

Importante: Los operadores de incremento y decremento aumentan o disminuyen en 1 respectivamente, ahora bien, existe una gran diferencia según la forma en que se utilicen, ya sea si colocamos el operador delante de la variable o después.

Post-incremento / Post-decremento, adiciona o reduce en 1 la variable, esta acción tiene efecto luego de finalizar la sentencia, mientras que el Pre-incremento / Pre-decremento tiene efecto en la misma sentencia adicionando o reduciendo en 1 la variable.

En C++ no existe un operador para la potencia, sin embargo existe una función en la librería <cmath> para esto: pow(base, exponente)

En la sección de ejemplos se encuentran distintos programas mostrando como funcionan todos los operadores mencionados en esta sección.

Operadores aritméticos

Ejemplo	Comentario
`int resultado = 10 % 2;`	La variable resultado es igual a cero pues lo que sobra de dividir 10 entre 2 es cero
`int c = 3;` `int alfa = c++;`	La variable alfa es igual a 3 pues se realiza un post-incremento
`int d = 0;` `int beta = --d;`	La variable beta es igual a -1 pues se realiza un pre-incremento

`double lado1 = 4.5;`
`double lado2 = 3.0;`
`double area = lado1 * lado2;`
	La variable area es igual a 13.5 pues es el resultado de multiplicar 4.5 por 3

`int cnt = 0;`
`int br = 2;`
`int omg = ++cnt * br++;`
	La variable omg es igual a 2 pues es el resultado de multiplicar 1 * 2. Si examinamos la expresión ++cnt es ejecutado en la misma línea, alterando el valor de cnt e incrementándolo en 1, mientras que br++ no produce cambio alguno en la misma línea ya que se utiliza un post-incremento y este tiene efecto al terminar la sentencia, o sea, br tendrá el valor de 3 en la línea siguiente después de la expresión.

Operadores aritméticos

Operador	Expresión	Equivalente	Comentario
+=	a = a + b	a += b	Le suma el valor de b al valor que tiene a y lo almacena en a
-=	a = a – b	a –= b	Le resta el valor de b al valor que tiene a y lo almacena en a
*=	a = a * b	a *= b	Multiplica el valor de b por el valor que tiene a y lo almacena en a
/=	a = a / b	a /= b	Divide el valor de a por el valor que tiene b y almacena el resultado en a
%=	a = a % b	a %= b	Determina cual es el residuo de dividir a por b y el resultado lo almacena a

Ejemplos:

```
int acum = 0;
int indice = 0;
int valor = 23;
acum += indice++ - valor; // acum = acum + (indice++ - valor)

long factorial = 1;
int numero = 5;
factorial *= numero; // factorial = factorial * numero

double sumDesc = 0;
double factura = 23500;
double desc = 6;
sumDesc += factura * (desc/100); // sumDesc = sumDesc + (factura * desc / 100)
```

Operadores de relación

Operador	Descripción
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
==	Igual que
!=	Distinto de

Ejemplos:

```
bool resultado;
double sueldo = 35000;
resultado = (sueldo > 34500); // true

bool seguir;
char opcion = 'N';
seguir = (opcion == 'S'); // false

bool continuar;
bool encontrado = false;
continuar = !encontrado; // true

bool intercambiar;
int a = 4;
int b = 2;
intercambiar = ( (--a * ++b) <= 12 ); // true

bool esMayor;
int num1 = 57;
int num2 = 79;
esMayor = (num1 > num2); // false

bool esDistinto;
char letra1 = 'Z';
char let5ra2 = 'X';
esDistinto = (letra1 != letra2); // true
```

Operadores lógicos

Operador	Descripción
!	Negación
&&	Conjunción (y)
||	Disyunción (o)

P	Q	P && Q (conjunción)
True	True	**True**
True	False	*False*
False	True	*False*
False	False	*False*

P	Q	P || Q (disyunción)
True	True	**True**
True	False	**True**
False	True	**True**
False	False	*False*

Concepto:

La conjunción (&&): solo retorna un valor de verdad verdadero (*true*) si ambos operándoos o condiciones que rodean el operador tienen como valor de verdad verdadero (*true*).

La disyunción (||): retorna un valor de verdad verdadero (*true*) siempre que haya por lo menos un operando o condición con valor de verdad verdadero (*true*), es decir, solamente es falsa cuando ambas condiciones lo son.

Ejemplos:

```
bool resultado;
resultado = true && false; // false

bool buscar = true;
bool encontrado = false;
bool resultado = buscar || encontrado; // true

bool terminar = false;
int capacidad = 34;
terminar = !(capacidad < 100); // false
```

Operador ()

- Los paréntesis nos permiten:
  - **Agrupar expresiones**
  - **Aislar expresiones condicionales**
  - **Indicar llamadas a funciones y argumentos o parámetros de funciones**
  - **Dar prioridad en la ejecución o evaluación dentro de una expresión**

Precedencia y asociatividad

Tabla de precedencia de operadores

Categoría	Operador	Asociatividad	Orden
Agrupación	`( )`	Izquierda a Derecha	1
Unarios	`+ - ! ++ --`	Derecha a Izquierda	2
Aritméticos	`* / %`	Izquierda a Derecha	3
Aritméticos	`+ -`	Izquierda a Derecha	4
Relacional	`< <= > >=`	Izquierda a Derecha	5
Igualdad	`== !=`	Izquierda a Derecha	6
(AND) lógico	`&&`	Izquierda a Derecha	7
(OR) lógico	`||`	Izquierda a Derecha	8
Condicional	`?:`	Derecha a Izquierda	9
Asignación	`= += -= *= /= %=`	Derecha a Izquierda	10

Concepto:

Una expresión puede estar compuesta por múltiples operándoos y diversos operadores, en tal sentido la expresión es evaluada de acuerdo al orden y asociatividad expresada en la tabla de precedencia de operadores.

La tabla muestra el orden de ejecución o evaluación de los operadores cuando son de categoría distinta y la asociatividad que deben seguir cuando pertenecen a la misma. Esto es, si en una expresión aparecen dos operadores de la misma categoría, la evaluación se realizará de acuerdo a lo expresado en la tabla.

Para ser más precisos, si en una expresión aparece una suma y una resta, la evaluación se hará de izquierda a derecha. Si en una expresión aparece una suma y una división, la división se hará primero sin importar donde se encuentre por que tiene precedencia según el orden de la tabla.

Precedencia y asociatividad

Ejemplos sobre precedencia de operadores:

$$s = \frac{a+b}{x-y} + \frac{n}{m+1}$$

```
s = ((a + b) / (x - y)) + (n / (m + 1))
```

$$z = \frac{x}{y+2}$$

```
z = x / (y + 2)
```

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
x1 = -b + sqrt(pow(b, 2.0) - 4 * a * c) / (2 * a)
x1 = -b - sqrt(pow(b, 2.0) - 4 * a * c) / (2 * a)
```

$$c = a^2 + b^2$$

```
c = pow(a, 2.0) + pow(b, 2.0)
```

Mezclar tipos de datos

	char	short	int	long	float	double
char	char	short	int	long	float	double
short	short	short	int	long	float	double
int	int	int	int	long	float	double
long	long	long	long	long	float	double
float	float	float	float	float	float	double
double	double	double	double	double	double	double

Concepto:

La tabla más arriba muestra la conversión automática que se lleva a cabo cuando en una expresión se ven involucrados tipos de datos distintos. Al final de la evaluación el resultado será convertido al tipo de dato de la variable a la cual será asignada dicho valor.

Ejemplos:

```
int a = 1;
short b = 2;
a = b - 15;
// b es promovido a
// int

float z;
float x = 5.0;
int y = 5;
z = x * y;
// y es promovido a
// float

long p = 54;
int s = 34;
double n;
n = p - s;
// s es promovido
// automaticamente a
// long antes de la
// resta y el
// resultado es a su
// vez convertido a
// double
```

Conversiones explícitas (type casting)

- Podemos optar por realizar conversiones de datos de forma explícita con el objetivo de modificar el tipo de dato ya sea para subir o bajar su capacidad

- Forma general:

```
static_cast<tipo>(valor);
```

- Donde:

  valor es una variable o literal

  tipo es el tipo de datos al que deseamos convertir el *valor* proporcionado

Conversiones explícitas (type casting)

- Ejemplos:

```cpp
#include <iostream>

using namespace std;

int main(int argc, char** argv)
{
    double num = 23.5702;

    int resultado = static_cast<int>(num);

    return 0;
}
```

Ejemplo 1:
Convertimos la variable *num* de *double* a *int*, y luego se asigna el valor a la variable *resultado*.
Almacenará 23

```cpp
#include <iostream>

using namespace std;

int main(int argc, char** argv)
{
    int sumEdades = 587;
    int cantidad = 8;

    double edadPromedio = static_cast<double>(sumEdades) / cantidad;

    return 0;
}
```

Ejemplo 2:
Convertimos la *sumEdades* de *int* a *double*, esto es para que al dividir *sumEdades* entre cantidad, el resultado pueda ser un número real, de lo contrario el resultado seria truncado, en otras palabras, sería un valor entero y deseamos obtener un valor con cifras decimales.
Almacenará 73.375

Conversiones explícitas (type casting)

- C++ ofrece también otras formas para realizar las conversiones explícitas

- Forma general:

```
(tipo) expresión;
expresión (tipo);
```

- Donde:

  expresión es un conjunto de operadores y operándoos que al ser evaluado retorna un valor; también puede ser una variable o literal

  tipo es el tipo de datos al que deseamos convertir el valor resultante de evaluar la *expresión*

Conversiones explícitas (type casting)

- Ejemplos:

```
double numero = 56.58902;
int numEntero = (int) numero; // almacenara 56


int sumEdades = 587;
int cantidad = 8;
double edadPromedio = (double) sumEdades / cantidad; // almacenara 73.375


float velocidad = 12.5267;
long resultado = long (velocidad); // almacenara 12
```

Contenido

- Elementos del lenguaje.
- Variables y tipos de datos.
- Operadores (aritméticos, de relación, lógicos).
- Ambiente de programación (compilador, herramientas).
- Análisis del problema.
- Mi primer programa en C++.
- Operaciones de entrada y salida.

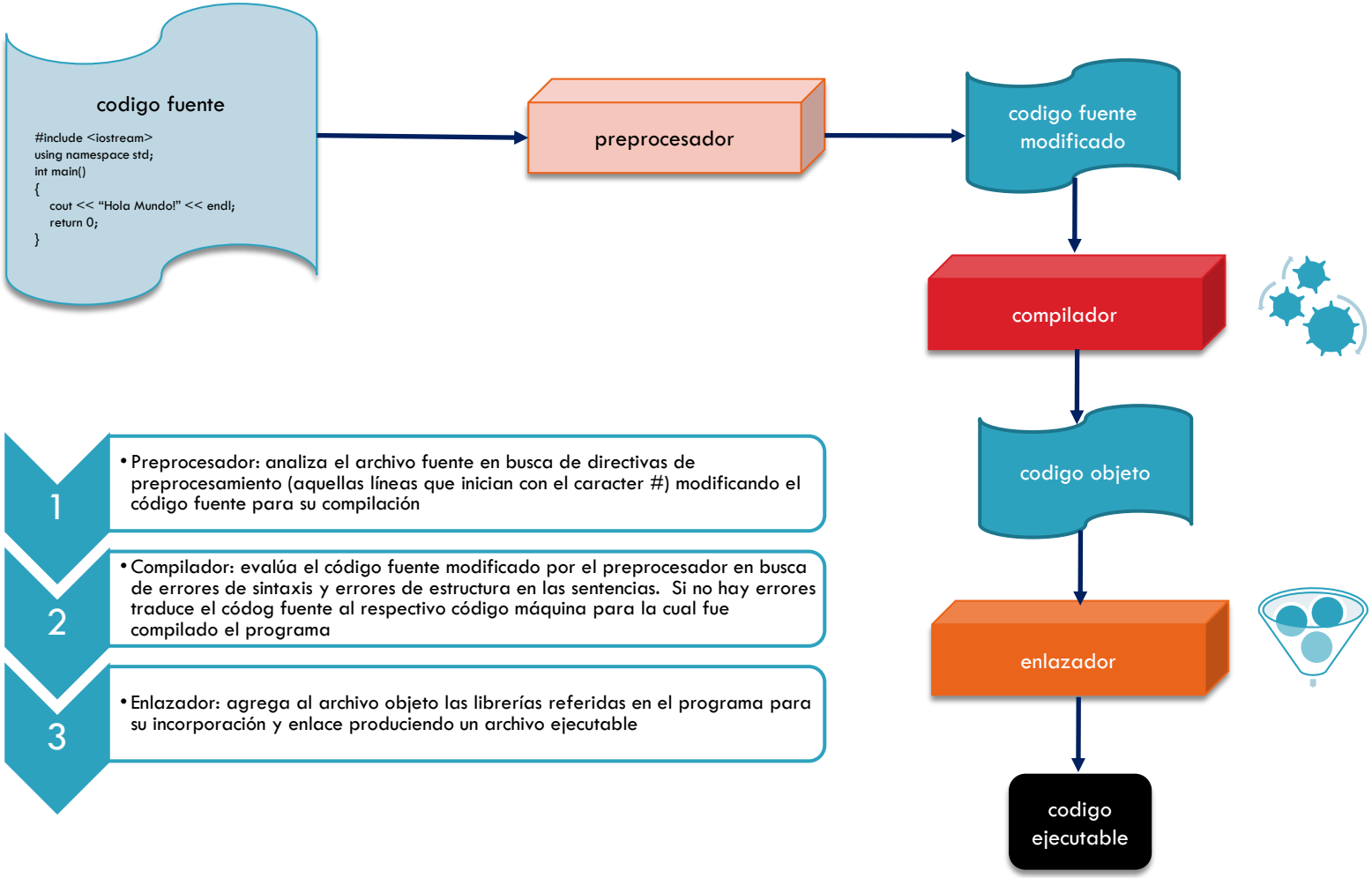
Ambiente de programación

Compilador, herramientas

Programa

- Un programa es un conjunto de instrucciones destinadas a seguir un flujo de pasos para resolver un problema o desempeñar algún tipo de tarea
- Código fuente: es el archivo que contiene el conjunto de instrucciones entendible para el ser humano (carece de un significado que pueda ser ejecutado por el procesador)
- Código objeto: es el resultado de analizar el código fuente para garantizar que no tenga ningún tipo de error en la escritura de las sentencias (sintaxis) o en su estructura (semántica)
- Código ejecutable: resultado de enlazar librerías que nuestro programa podría utilizar para producir un programa ejecutable de acuerdo a la plataforma para la cual fue compilado

Compilación y enlazado



Compilador, IDE y SDK

- Un *compilador* es el software utilizado para convertir el código fuente de un programa a un lenguaje que pueda ser interpretado por el procesador de un computador (código máquina)
- *IDE* o Ambiente de Desarrollo Integrado por sus siglas en inglés (Integrated Development Environment) es un software que permite el desarrollo de software utilizando un lenguaje de programación (no incluye un compilador). Un IDE consta de diversas herramientas para depuración de programas, autocompletación de sentencias, editor avanzado, etc.
- *SDK* (Software Development Kit) es un conjunto de herramientas de producción de software que permite al desarrollador crear aplicaciones para una determinada plataforma. En muchos casos *incluye el IDE*

Ejemplos de IDE para C++

- Eclipse http://www.eclipse.org
- Netbeans http://www.netbeans.org
- Bloodshed Dev-C++ http://www.bloodshed.net
- BVRDE http://bvrde.sourceforge.net
- CodeBlocks http://codeblocks.org (puede ser descargado con el compilador de C++ MinGW incluido para ambiente Windows, para ambiente Linux recomendamos realizar la instalación por separado, es decir, instalar C++ y descargar el IDE de forma independiente)

Ejemplos de SDK para C++

- Eclipse SDK
- Qt
- Microsoft Visual Studio .NET
- Valentina C++
- C++ Builder

Compilador utilizado en este curso

- El compilador que utilizaremos será C++ / GNU. Este compilador se encuentra disponible gratuitamente para plataformas Windows, Linux y Mac
- En ambiente Windows este compilador puede ser encontrado bajo el nombre de MinGW (Minimalist GNU for Windows) http://www.mingw.org/
- Inicialmente estaremos trabajando con compilación y ejecución de programas desde la línea de comando o terminal y más adelante incorporaremos el uso de un IDE (CodeBlocks en nuestro caso)

Como compilamos un programa

- Desde la línea de comando en ambiente Windows:

```
C:\> g++ programa.cpp -o programa
```

- Desde la terminal en ambiente Linux:

```
$ g++ programa.cpp -o programa
```

- Donde:
  - programa.cpp el archivo fuente con código C++ dentro, este archivo debe ser editado en cualquier editor de texto y salvado con extensión cpp
  - programa es el nombre del archivo ejecutable, el resultado de compilar programa.cpp (puede ser otro nombre distinto al archivo fuente)
  - g++ es el comando utilizado para la compilación y enlazado de nun programa fuente
  - -o es la opción que indica el nombre del archivo ejecutable

Contenido

- Elementos del lenguaje.
- Variables y tipos de datos.
- Operadores (aritméticos, de relación, lógicos).
- Ambiente de programación (compilador, herramientas).
- Análisis del problema.
- Mi primer programa en C++.
- Operaciones de entrada y salida.

Análisis del problema

¿Cómo abordar un problema para buscar su solución?

El lenguaje de programación

- El objetivo del aprendizaje de un lenguaje de programación es ser más eficaz en el diseño e implementación de nuevos sistemas y en el mantenimiento de los viejos
- La apreciación de las técnicas de diseño y programación es mucho más importante que el total entendimiento de los detalles; ese entendimiento viene con la práctica y el tiempo
- ¿Qué es construir una aplicación C++? Construir una aplicación C++ es transformar el algoritmo de la solución de un problema en un programa ejecutable

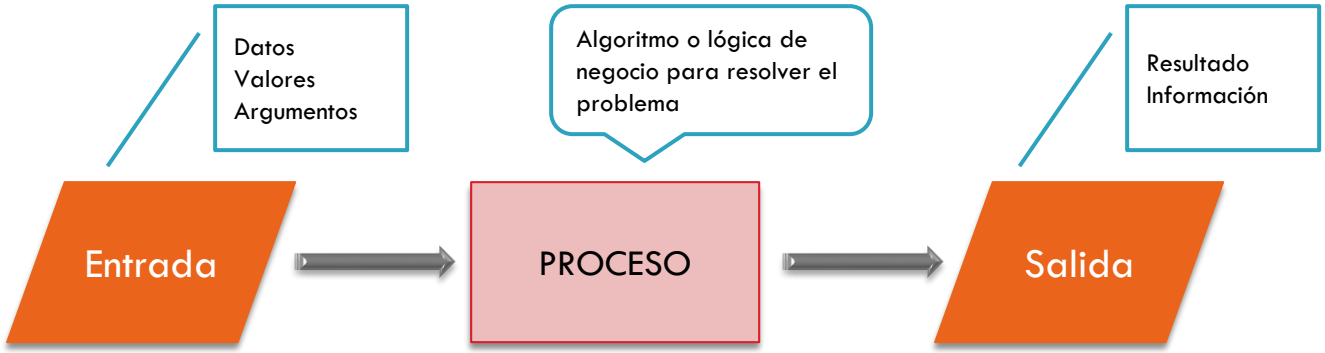
Importante:
Lo indicado anteriormente implica que ya se cumple con el pre-requisito de saber pasar del problema a la escritura del algoritmo ya sea utilizando pseudocódigo o flujo gramas.

Cómo abordar un problema

- La forma de abordar un problema para buscar su solución es muy personal, no existe un libro o fórmula que nos indique como plantear paso-a-paso un algoritmo que resuelva un problema; esto se pudo experimentar en la asignatura anterior mediante la confección de algoritmos con diagramas de flujo o pseudocódigos
- Sin embargo, para diseñar un algoritmo es imprescidible conocer primero como solucionar el problema de forma no automatizada, es decir, sin la intervención de un programa. Es necesario comprender con plena claridad lo que se busca o necesita lograr y como hacerlo de forma manual; todo esto implica que debe realizar un análisis que lo lleve a tener un dominio del problema

Enfoque

- Se hace necesario que pueda plantear su diseño desde una perspectiva de Entrada / Proceso / Salida
- Este enfoque le permitirá identificar que necesita producir u obtener (salida), que elementos necesita para producir dicho resultado (entrada) y cómo debe hacerlo (proceso)

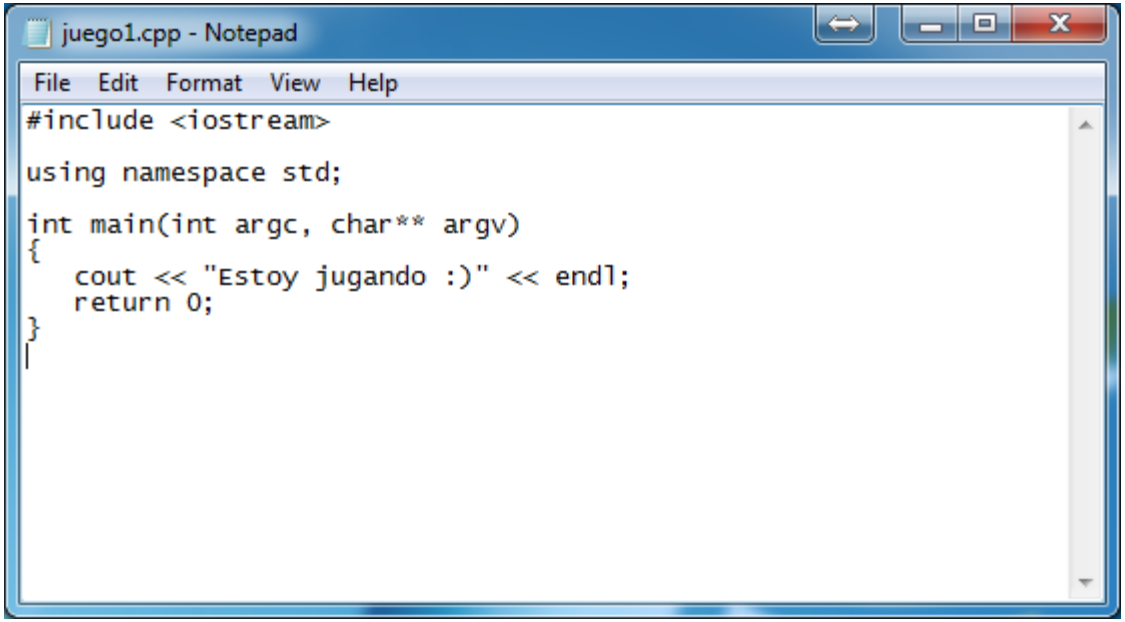


Diseño del algoritmo

- Una buena práctica a nivel inicial consiste en escribir el algoritmo en pseudocódigo o flujo gramas antes de escribir el código fuente en C++; incluso puede realizar pruebas de escritorio que le permitan probar si su algoritmo funciona de acuerdo a lo esperado
- Con la práctica ya no sentirá la necesidad de apoyarse en estas herramientas didácticas y prescindirá de ellas, abordando el problema directamente utilizando el lenguaje de programación

Pasos para construir una aplicación

1. Escribir el código fuente utilizando un editor de texto ASCII:
   - ✓ Notepad, Notepad++, Ultraedit, etc. (Windows)
   - ✓ vi, gEdit, Geany, etc. (Linux)

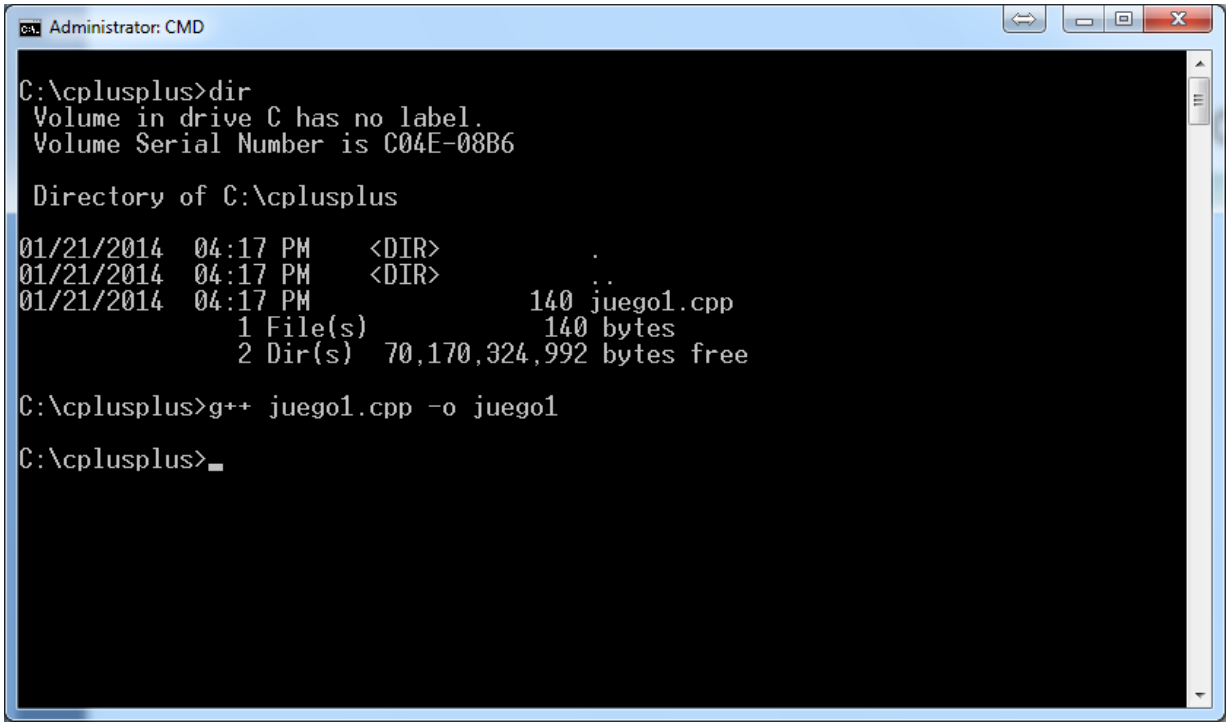


Pasos para construir una aplicación

2. Grabar como archivo de texto el código digitado, el nombre del archivo debe llevar extensión cpp, es decir, si queremos nombrar a nuestro programa como juego1, debemos gragar el archivo como juego1.cpp Es recomendable que tenga un directorio de trabajo donde almacene todos sus programas fuentes, de forma que sea fácil ubicarlos cada vez que necesite trabajar con ellos (por ejemplo: `c:\cplusplus` o `c:\cursocpp` en Windows, `/home/usuario/cplusplus` o `/home/usuario/cursocpp` en Linux)

Pasos para construir una aplicación

3. Compilar el archivo:
   - ✓ `C:\cplusplus> g++ juego1.cpp -o juego1` (en ambiente Windows)
   - ✓ `$ g++ juego1.cpp -o juego1` (en ambiente Linux)

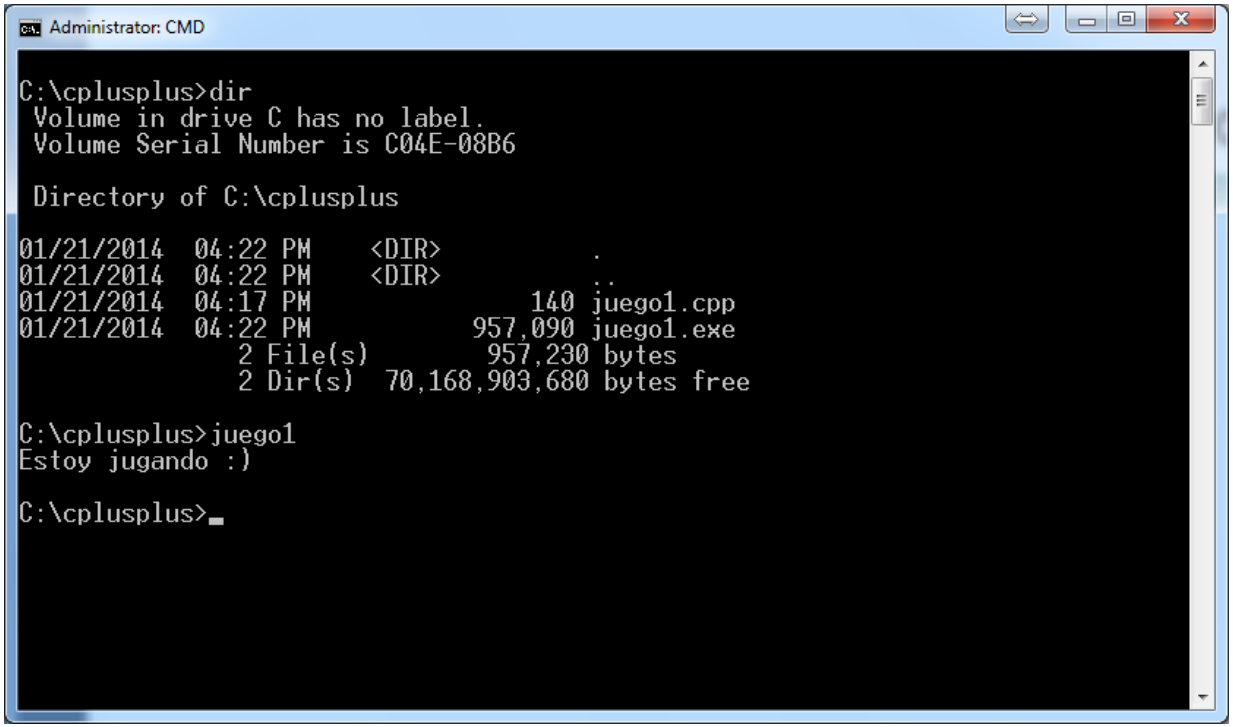


Pasos para construir una aplicación

4. El paso anterior realiza el proceso de compilación, se ejecutarán las directivas de pre-procesamiento (aquellas líneas que inician con #), y finalmente se produce el proceso de enlazado, el archivo reslultante será un archivo ejecutable con el mismo nombre de nuestro programa fuente si así fue especificado con la opción -o
5. Si durante este proceso, el compilador encuentra algún error de sintaxis o algo que viole las reglas del lenguaje, el archivo ejecutable no sera producido y obtendremos por pantalla el listado de los errores encontrados, es importante familiarizarnos con la forma en que cada compilador informa las advertencias y los errores encontrados

Pasos para construir una aplicación

6. Ejecutar el programa:
   - ✓ `C:\cplusplus> juego1 [Enter]` (en ambiente Windows)
   - ✓ `$ ./juego1 [Enter]` (en ambiente Linux)

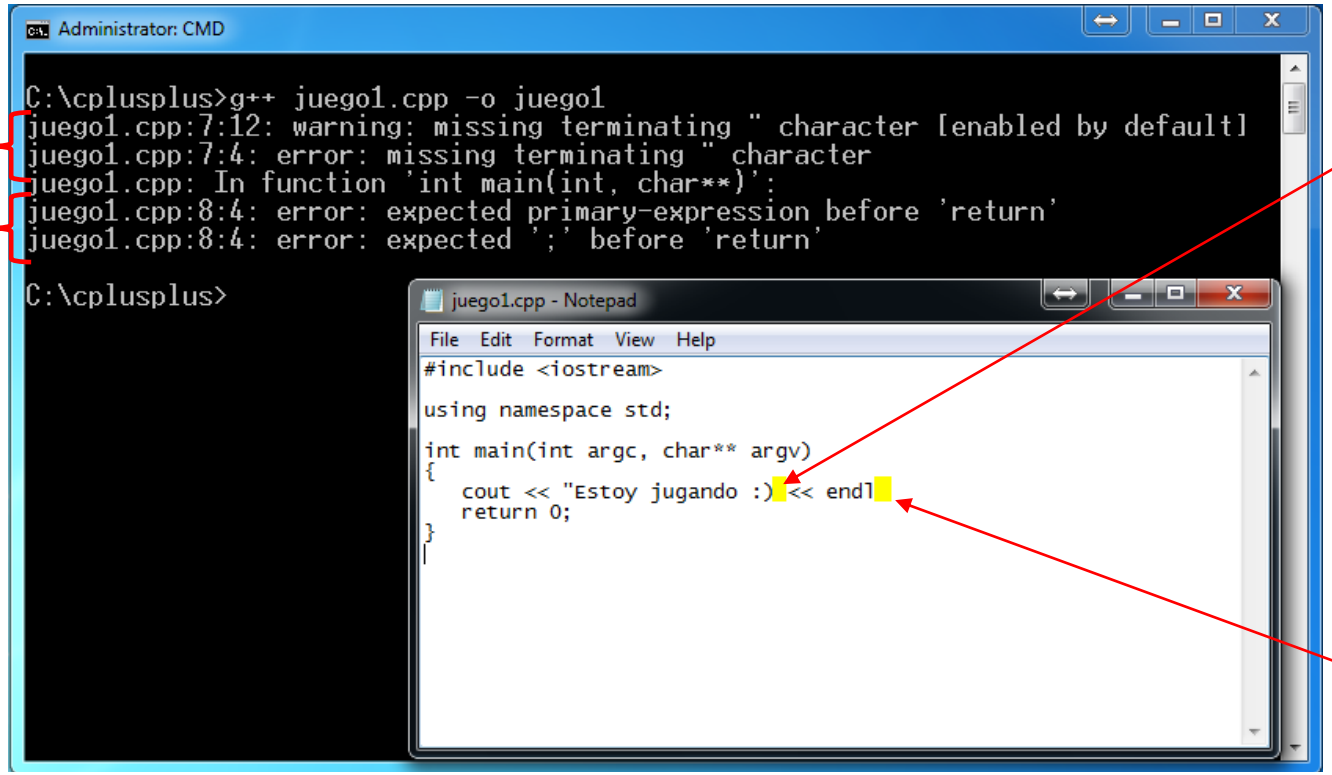


Pasos para construir una aplicación

7. Probar nuestra aplicación. Una vez hayamos logrado producir una compilación exitosa, debemos probar nuestro programa para determinar si hace o se comporta como se espera. De existir algún problema en la ejecución, ya sea que nuestro programa arroje un resultado incorrecto o no pueda realizar la tarea para el cual fue escrito entonces debemos volver a nuestro programa fuente para corregir el problema (estos son errores de lógica)

Pasos para construir una aplicación

- Cuando nuestro programa tiene errores de sintaxis:



Otras opciones de compilación

- Compilación sin enlazado (no produce archivo ejecutable)

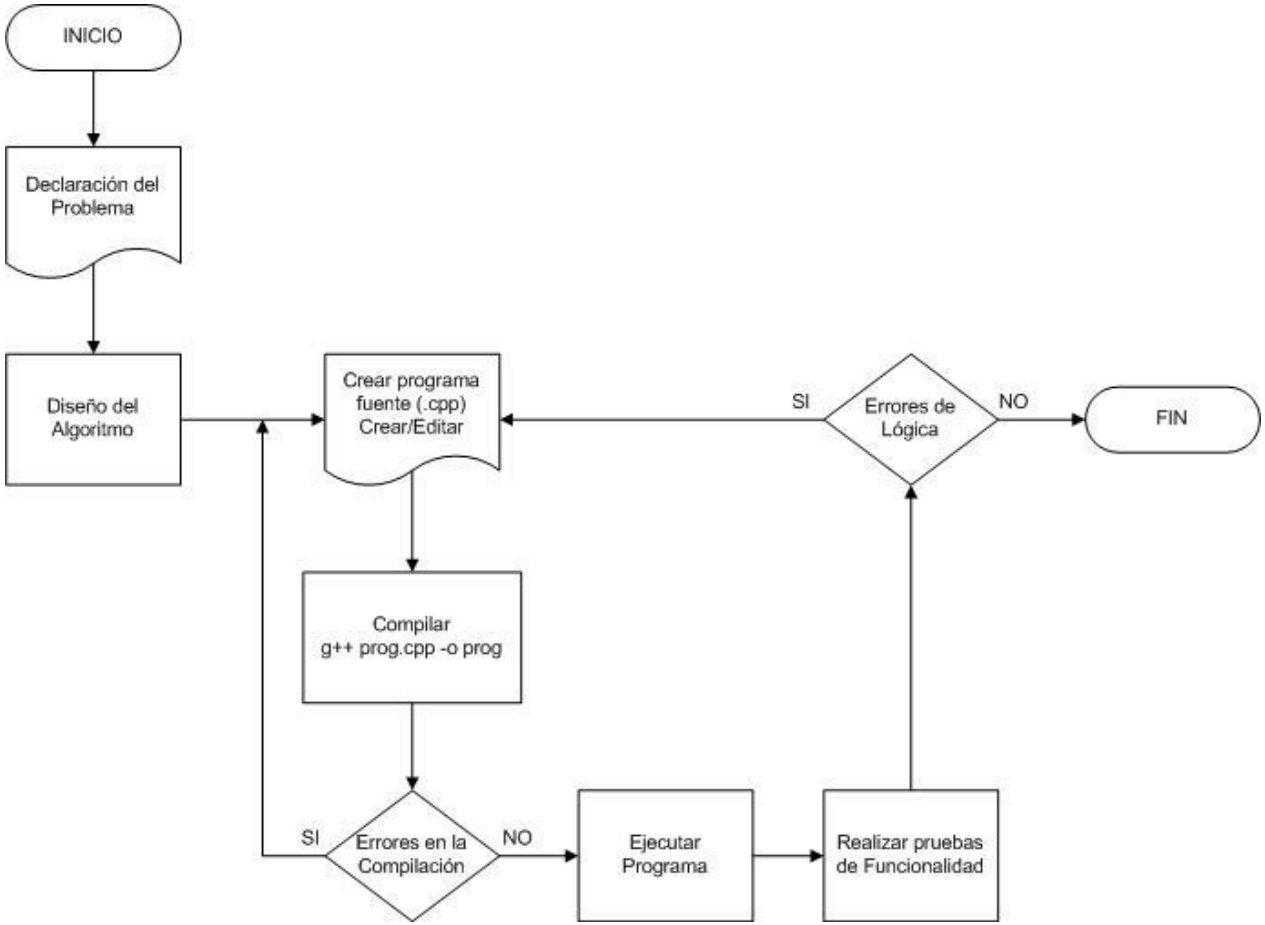
```
g++ -c juego1.cpp –o juego1.o
```

- Enlazado cuando ya tenemos el archivo objeto

```
g++ -o juego1.exe juego1.o
```
(en ambiente Windows)

```
g++ -o juego1 juego1.o
```
(en ambiente Linux)

Diagrama de pasos (resumen)



Contenido

- Elementos del lenguaje.
- Variables y tipos de datos.
- Operadores (aritméticos, de relación, lógicos).
- Ambiente de programación (compilador, herramientas).
- Análisis del problema.
- Mi primer programa en C++.
- Operaciones de entrada y salida.

Mi primer programa en C++

Hola Mundo!!

Mi primer programa

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       cout << "Hola Mundo!!" << endl;
8       return 0;
9   }
10
```

Esta línea es una directiva de pre-procesamiento e indica que nuestro programa incluirá la librería ***iostream*** (*input-output stream*) la cual contiene diversas funciones relacionadas a la entrada y salida de data, siempre que utilicemos el objeto ***cout*** debemos incluir esta línea en nuestros programas.

Sin esta especificación, nuestro programa tendría error de compilación ya que C++ no podría localizar el objeto ***cout*** referido en la línea 7 de nuestro programa.

C++ tiene una gran variedad de identificadores nativos cuyos nombres están almacenados en un lugar llamado espacio de nombre estándar (***namespace std***). Para acceder a estos nombres hay que indicárselo al compilador. La razón de los espacios de nombres (namespace) es evitar conflicto entre nombres en diferentes partes de un programa.

Mi primer programa

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       cout << "Hola Mundo!!" << endl;
8       return 0;
9   }
10
```

Esta línea despliega el mensaje *"Hola Mundo!!"* por pantalla, adicionalmente también despliega un carácter de nueva línea (***endl***), esto es, después de mostrar los caracteres entre comillas dobles, agrega a esta acción de "mostrar" una nueva línea. Para esto utilizamos el objeto ***cout*** separando cada elemento que deseamos enviar a pantalla por el operador **<<** *y al final un punto y coma para indicar el final de la sentencia.*

Los caracteres encerrados entre comillas comprenden un literal de cadena o literal string.

Esta es la línea inicial de la función ***main***, la cual es el punto de arranque de todo programa C++. Para que un programa o aplicación en C++ pueda ser ejecutado directamente necesita contener esta función especial. Como toda función, la misma cuenta con un cuerpo el cual es el que se encuentra encerrado entre llaves.

Piense en una función como una porción de programa que puede ser ejecutada para realizar un proceso la cual puede o no retornar un valor, en nuestro ejemplo, la función ***main*** está definida de forma que retorna un valor tipo ***int***. Además hay otros elementos encerrados entre paréntesis, por el momento no debemos preocuparnos por su significado, más adelante entenderemos para que se utilizan.

Otra forma de escribir la función ***main*** es obviando el contenido entre paréntesis: ***int main()***

Mi primer programa

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       cout << "Hola Mundo!!" << endl;
8       return 0;
9   }
10
```

Esta última línea indica el final de la función ***main.*** Como habíamos expresado anteriormente, nuestra función esta definida para retornar un valor tipo entero, por esta razón utilizamos la sentencia ***return*** que indique el final de nuestro código, especificando adicionalmente un valor entero (cero en nuestro ejemplo, usualmente este es el valor utilizado).

El uso de ***return*** es opcional, no obstante debe quedar claro que la función estaría retornando un valor basura.

Si la función retorna entero, debemos utilizar un valor entero en la sentencia ***return***. Puede ser un literal, variable o expresión.

Más sobre namespace std

- En algunos programas C++ podría encontrar la ausencia de la sentencia ***using***:

```
1  #include <iostream>
2
3  int main(int argc, char** argv)
4  {
5      std::cout << "Hola Mundo!!" << std::endl;
6      return 0;
7  }
```

- También podría encontrar su utilización en una forma distinta, haciendo referencia a cada objeto utilizado dentro del nombre de espacio ***std***:

```
1  #include <iostream>
2
3  using std::cout;
4  using std::endl;
5
6  int main(int argc, char** argv)
7  {
8      cout << "Hola Mundo!!" << endl;
9      return 0;
10 }
```

Más sobre namespace std

- Para evitar el estilo poco estético y confuso de usar:

```
using std::cout;
using std::cin;
using std::endl;
```

o

```
std::cout << …;
std::cin >> …;
std::endl;
```

- **Use "using namespace std;"** como lo verá en los códigos de ejemplo contenidos en todo este curso.

Más sobre namespace std

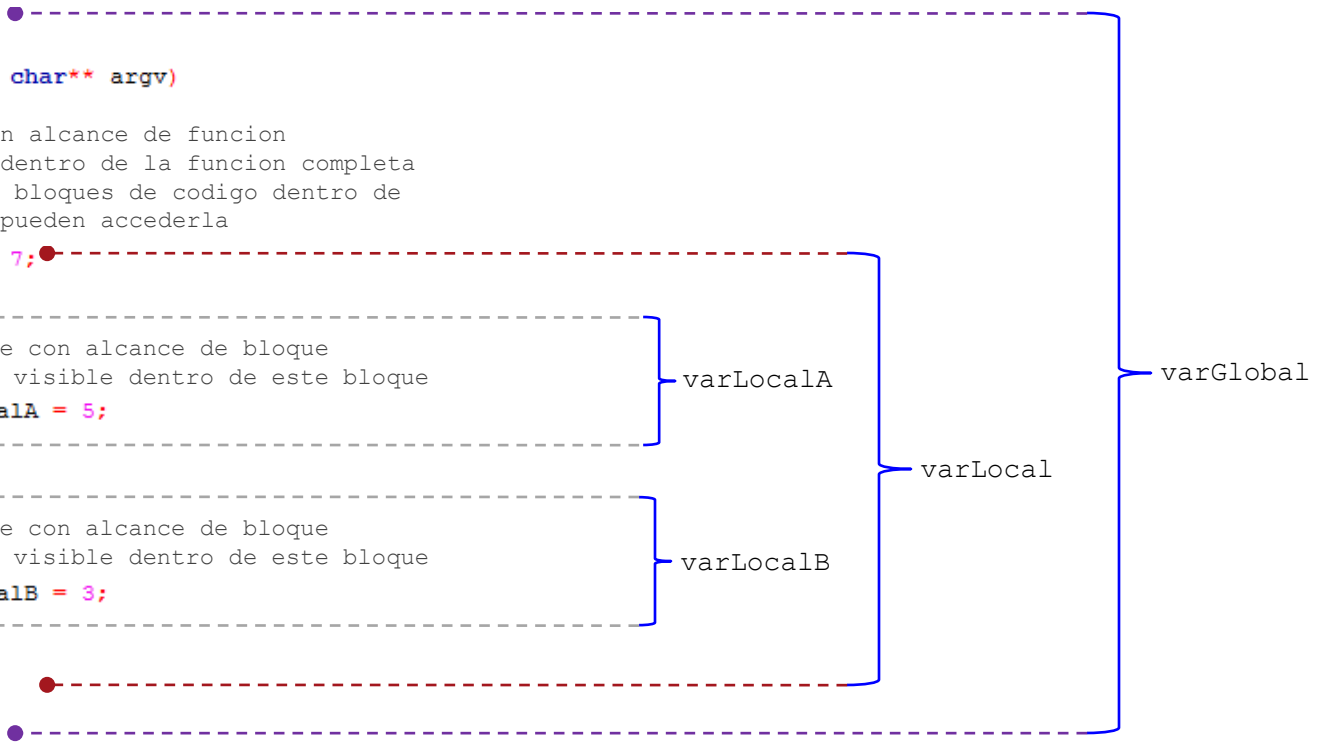
- En los compiladores de C++ estandarizados a la última versión, es mandatorio el uso de "***using namespace std;***" en los archivos .cpp inmediatamente después de las directivas de pre-procesamiento (los ***includes***):

```
#include <iostream>
#include <cmath>
using namespace std;
```

- Hacerlo así, es un buen estilo de programación C++.

Alcance de las variables (ejemplo)

```
1  #include <iostream>
2
3  using namespace std;
4
5  // Variable con alcance global
6  // es visible dentro del archivo completo
7  // todas las funciones y bloques pueden
8  // accederla
9  int varGlobal = 9;
10
11 int main(int argc, char** argv)
12 {
13     // Variable con alcance de funcion
14     // es visible dentro de la funcion completa
15     // y todos los bloques de codigo dentro de
16     // la funcion pueden accederla
17     int varLocal = 7;
18
19     {
20         // Variable con alcance de bloque
21         // solo es visible dentro de este bloque
22         int varLocalA = 5;
23     }
24
25     {
26         // Variable con alcance de bloque
27         // solo es visible dentro de este bloque
28         int varLocalB = 3;
29     }
30
31     return 0;
32 }
```



Alcance de las variables (ejemplo)

```
1  #include <iostream>
2
3  using namespace std;
4
5  // Variable con alcance global
6  // es visible dentro del archivo completo
7  // todas las funciones y bloques pueden
8  // accederla
9  int varGlobal = 9;
10
11 int main(int argc, char** argv)
12 {
13     // Variable con alcance de funcion
14     // es visible dentro de la funcion completa
15     // y todos los bloques de codigo dentro de
16     // la funcion pueden accederla
17     int varLocal = 7;
18
19     {
20         // Variable con alcance de bloque
21         // solo es visible dentro de este bloque
22         int varLocalA = 5;
23     }
24
25     cout << varLocalA << endl; /*** ERROR DE COMPILACION ***/
26
27     {
28         // Variable con alcance de bloque
29         // solo es visible dentro de este bloque
30         int varLocalB = 3;
31     }
32
33     return 0;
34 }
```

**** ERROR ****

Esta línea de código genera un error de compilación debido a la que la variable ***varLocalA*** solo es accesible dentro del bloque de código donde la misma fue definida.

Alcance de las variables (ejemplo)

```
1  #include <iostream>
2
3  using namespace std;
4
5  // Variable con alcance global
6  // es visible dentro del archivo completo
7  // todas las funciones y bloques pueden
8  // accederla
9  int varGlobal = 9;
10
11 int main(int argc, char** argv)
12 {
13     // Variable con alcance de funcion
14     // es visible dentro de la funcion completa
15     // y todos los bloques de codigo dentro de
16     // la funcion pueden accederla
17     int varLocal = 7;
18
19     {
20         // Variable con alcance de bloque
21         // solo es visible dentro de este bloque
22         int varLocalA = 5;
23     }
24
25     cout << varGlobal << endl; /*** OK ***/
26
27     {
28         // Variable con alcance de bloque
29         // solo es visible dentro de este bloque
30         int varLocalB = 3;
31         cout << varLocal << endl;  /*** OK ***/
32     }
33
34     return 0;
35 }
```

**** OK ****

La variable ***varGlobal*** es perfectamente visible desde cualquier punto del archivo fuente, por lo que puede ser referida en cualquier línea del programa.

**** OK ****

La variable ***varLocal*** puede ser accedida sin problemas desde cualquier punto de la función ***main*** pues fue declarada dentro de las llaves que encierran el cuerpo de dicha función.

Alcance de las variables (ejemplo)

```
1  #include <iostream>
2
3  using namespace std;
4
5  // Variable con alcance global
6  // es visible dentro del archivo completo
7  // todas las funciones y bloques pueden
8  // accederla
9  int varGlobal = 9;
10
11 int main(int argc, char** argv)
12 {
13     cout << varLocal << endl; /*** ERROR DE COMPILACION ***/
14     // Variable con alcance de funcion
15     // es visible dentro de la funcion completa
16     // y todos los bloques de codigo dentro de
17     // la funcion pueden accederla
18     int varLocal = 7;
19
20     {
21         // Variable con alcance de bloque
22         // solo es visible dentro de este bloque
23         int varLocalA = 5;
24     }
25
26     {
27         // Variable con alcance de bloque
28         // solo es visible dentro de este bloque
29         int varLocalB = 3;
30         cout << varLocalA << endl;  /*** ERROR DE COMPILACION ***/
31     }
32
33     return 0;
34 }
```

**** ERROR ****

A pesar de que la referencia que se hace la variable ***varLocal*** se encuentra dentro de las llaves de la función ***main,*** este referimiento se hace antes de que la variable sea declarada, por lo que se considera un error de '*out of scope*' o que la variable está fuera de alcance.

**** ERROR ****

La variable ***varLocalA*** no puede ser referida dentro del segundo bloque de código, pues está fuera de alcance, debido a que la misma fue declarada en el primer bloque solamente puede ser referida en ese 'scope' o nivel de alcance.

Alcance de las variables (ejemplo)

```
1  #include <iostream>
2
3  using namespace std;
4  // Variable con alcance global
5  // es visible dentro del archivo completo
6  // todas las funciones y bloques pueden
7  // accederla
8
9  int varGlobal = 9;
10
11 int main(int argc, char** argv)
12 {
13     // Variable con alcance de funcion
14     // es visible dentro de la funcion completa
15     // y todos los bloques de codigo dentro de
16     // la funcion pueden accederla
17     int varLocal = 7;
18     int varLocalA = 100;
19
20     {    // Variable con alcance de bloque
21          // solo es visible dentro de este bloque
22
23         int varLocalA = 5;
24         cout << varLocalA << endl; /*** OK: Despliega 5 ***/
25     }
26
27     cout << varLocalA << endl; /*** OK: Despliega 100 ***/
28
29     {
30          // Variable con alcance de bloque
31          // solo es visible dentro de este bloque
32         int varLocalB = 3;
33     }
34
35     return 0;
36 }
```

** OK **

Como podemos observar, hay dos declaraciones de la variable ***varLocalA***, una a nivel de la función ***main*** y otra a nivel del primer bloque de código de dicha función. Esto es perfectamente posible y en memoria se manejan como dos variables independientes. Esto significa que dentro del bloque de código, el valor de la variable es uno y fuera del bloque, otro.

La mejor forma de ilustrarlo es asumiendo que en lugar de utilizar el nombre ***varLocalA***, utilicemos ***alfa*** como variable a nivel de la función ***main***, y ***beta*** como variable a nivel del primer bloque de código, en tal sentido, visualmente dentro del programa fuente se comprende mejor que se trata de dos variables distintas.

Contenido

- Elementos del lenguaje.
- Variables y tipos de datos.
- Operadores (aritméticos, de relación, lógicos).
- Ambiente de programación (compilador, herramientas).
- Análisis del problema.
- Mi primer programa en C++.
- Operaciones de entrada y salida.

Operaciones de entrada y salida

cout <<

cin >>

endl

Operaciones de entrada y salida

- En C++ podemos hacer operaciones de entrada (leer) y salida (escribir) desde y hacia:
  - Dispositivos estándar de E/S (teclado / pantalla)
  - Archivos en dispositivos de almacenamiento (discos)
  - Cadenas de caracteres
- En esta parte del curso trabajaremos con operaciones de E/S desde y hacia los dispositivos estándar (teclado / pantalla)
- **Sentencias de salida:** permiten a nuestro algoritmo mostrar el resultado del procesamiento de los datos
- **Sentencias de entrada:** permiten a nuestro algoritmo interactuar con el exterior, capturando los datos a ser procesados

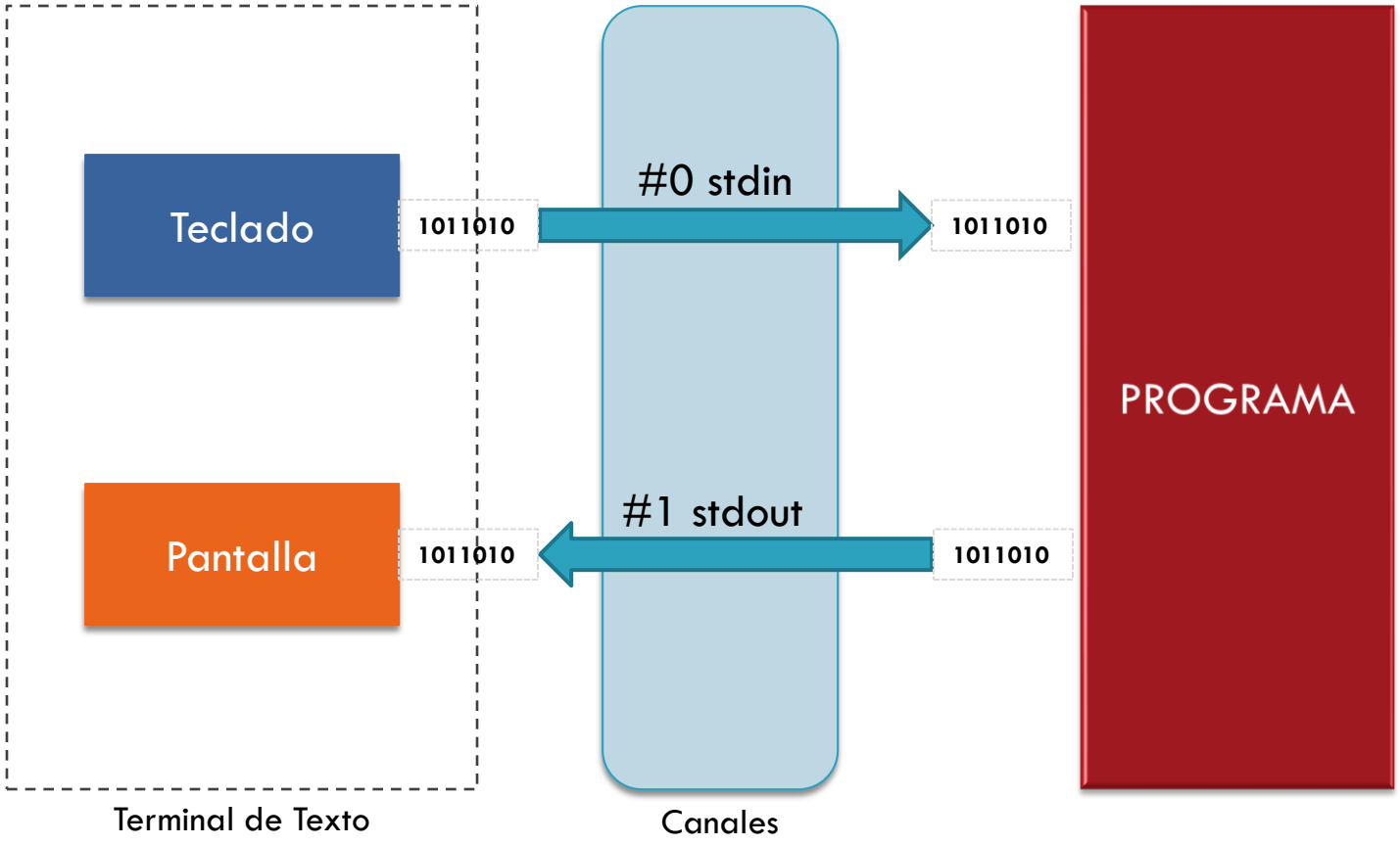
Flujo de bytes (stream)

- C++ gestiona las operaciones de E/S usando el concepto de flujo de bytes (bytes stream)
- Un flujo es una abstracción para referirse a cualquier flujo de datos desde una fuente (dispositivo de entrada) hacia un consumidor (dispositivo de salida)
- El concepto de flujo de bytes es una abstracción que permite que las operaciones de E/S sean vistas por el programador de manera similar e independiente de los elementos de bajo nivel del sistema operativo (controladores, conexión con los dispositivos de E/S, etc.)
- En el modelo de flujo, la información se mueve como secuencia de bytes anónimos que entran por el flujo de entrada y salen por el flujo de salida

Flujo de bytes (stream)

- La información es *insertada* en el flujo de salida o *extraída* desde el flujo de entrada
- Extracción, obtener y recoger son sinónimos cuando se habla de entrada de bytes desde una **fuente** (dispositivo de entrada)
- Inserción, poner y almacenar son sinónimos cuando se habla de salida de bytes hacia un **consumidor** (dispositivo de salida)
- Entrada: un flujo desde un dispositivo de entrada (teclado, disco, scanner, red) hacia la memoria principal
- Salida: un flujo desde la memoria principal hacia un dispositivo de salida (pantalla, impresora, red, disco, pendrive)

Flujos de E/S estándar



Flujos de E/S estándar

- Los estándar son canales de entrada y salida pre-conectados al programa y su entorno (normalmente un terminal de *texto compuesto* por un *teclado* y una *pantalla*) cuando este inicia su ejecución
- Los canales pre-conectados son tres y son llamados:
  - #0 standard input, stdin
  - #1 standard output, stdout
  - #2 standard error, stderr
- En este curso trabajaremos básicamente con los flujos estándar stdin y stdout

Recursos para E/S estándar

- Para las operaciones de E/S por los dispositivos estándar (teclado y pantalla), C++ dispone de los recursos:
  - `cin >>`
  - `cout <<`
  - `endl`
- Todos estos recursos se encuentran en la librería `<iostream>`

Recursos para E/S estándar

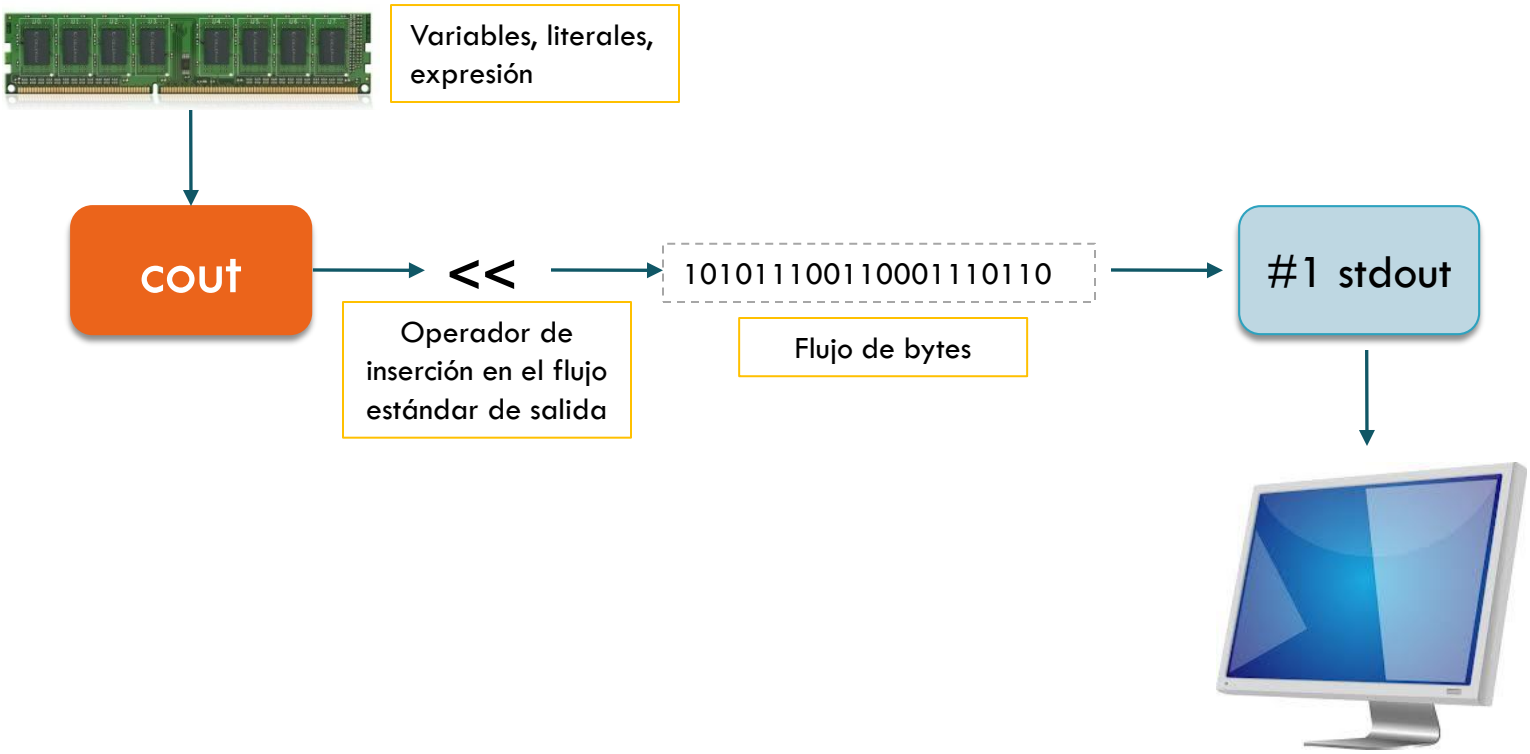
- `cout <<` : para ***insertar*** bytes de información en el flujo de salida hacia el dispositivo por defecto (pantalla) usando el operador de inserción `<<`
- `cin >>` : para ***extraer*** bytes de información desde el flujo de entrada hacia una variable (vía el dispositivo por defecto, teclado) usando el operador de inserción `>>`
- `endl` : es un manipulador de flujo que se utiliza con `cout <<` ; inserta un carácter de nueva línea (CR/LF) y vacía el buffer de salida; *es preferible su uso a "\n" el cual solo inserta un newline o caracter de nueva línea*

cout <<

- `cout << expresión;`
- Donde expresión es evaluada y su resultado es entonces ***insertado*** en el flujo estándar de salida para ser mostrado en el dispositivo por defecto, la pantalla

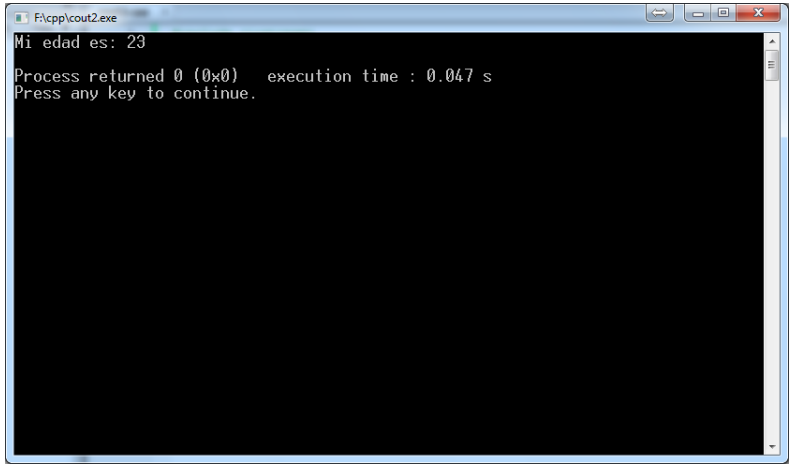
- `cout << expresión << endl;`
- Donde expresión es evaluada y su resultado es entonces ***insertado*** en el flujo estándar de salida y almacenado en el búffer para ser mostrado su contenido en el dispositivo por defecto, la pantalla, con un caracter de nueva línea al final

De RAM a la pantalla



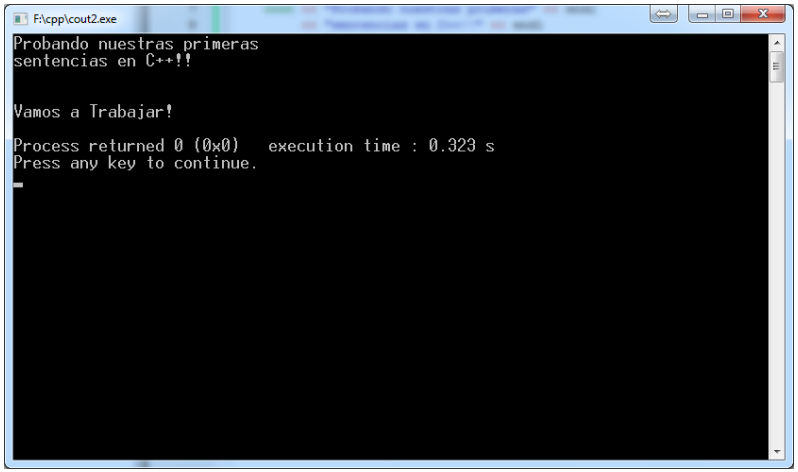
Ejemplos con cout

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      int edad = 23;
8
9      cout << "Mi edad es: " << edad << endl;
10
11     return 0;
12 }
```



Ejemplos con cout

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      cout << "Probando nuestras primeras" << endl
8           << "sentencias en C++!!" << endl
9           << "\n\nVamos a Trabajar!" << endl;
10
11     return 0;
12 }
```



Ejemplos con cout

```cpp
#include <iostream>

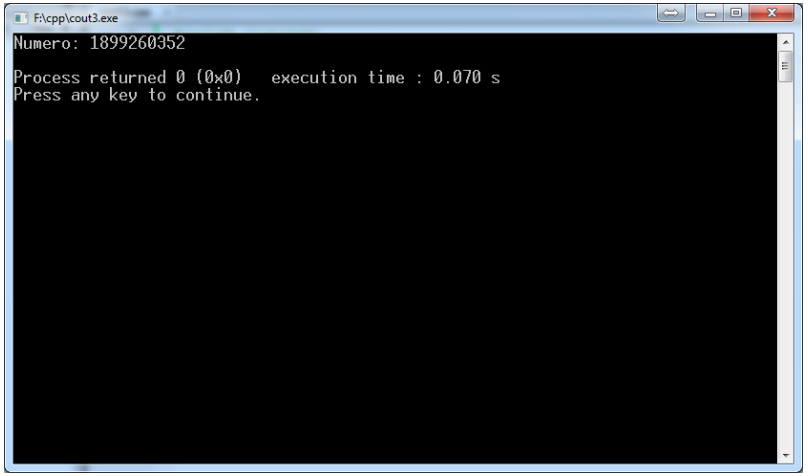
using namespace std;

int main(int argc, char** argv)
{
    int num1 = 32568536;
    int num2 = 25632552;

    long res = static_cast<long>(num1) * num2;

    cout << "Numero: " << res << endl;

    return 0;
}
```



cin >>

- `cin >> variable;`
- La ejecución del programa se detiene a esperar una entrada suplida por el teclado, ignorando los caracteres de espacio en blanco (barra espaciadora, tab, enter) que hayan sido pulsados antes
- En ***variable*** es que se captura la información, por lo tanto, el tipo debe ser correspondiente con el tipo de dato a capturar
- Al terminar la captura (extracción) `cin` agrega un *newline* en el flujo estándar de entrada
- La extracción se realiza hasta el primer carácter de espacio en blanco encontrado

Del teclado a RAM



Variables, literales, expresión

cin

>>

Operador de extracción del flujo estándar de entrada

10101110011000111010

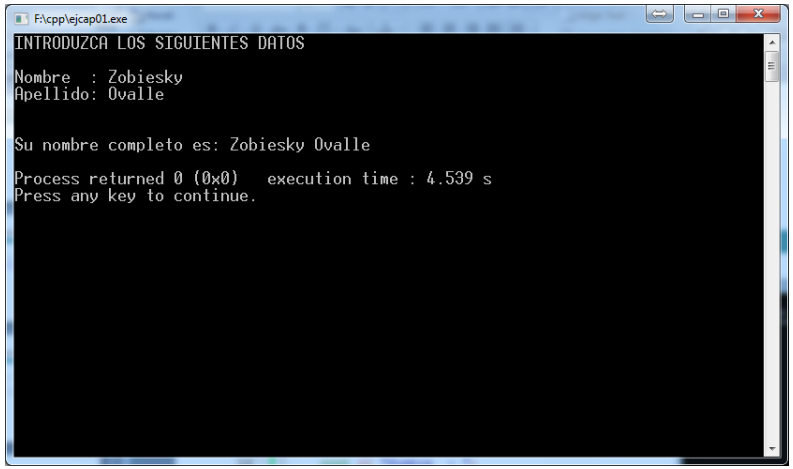
Flujo de bytes

#0 stdin



Ejemplos con cin y cout

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       char nombre[20];
8       char apellido[20];
9
10      cout << "INTRODUZCA LOS SIGUIENTES DATOS"
11           << endl << endl;
12      cout << "Nombre   : ";
13      cin >> nombre;
14      cout << "Apellido: ";
15      cin >> apellido;
16
17      cout << "\n\nSu nombre completo es: "
18           << nombre << " " << apellido
19           << endl;
20
21      return 0;
22  }
```



Ejemplos con cin y cout

```cpp
#include <iostream>

using namespace std;

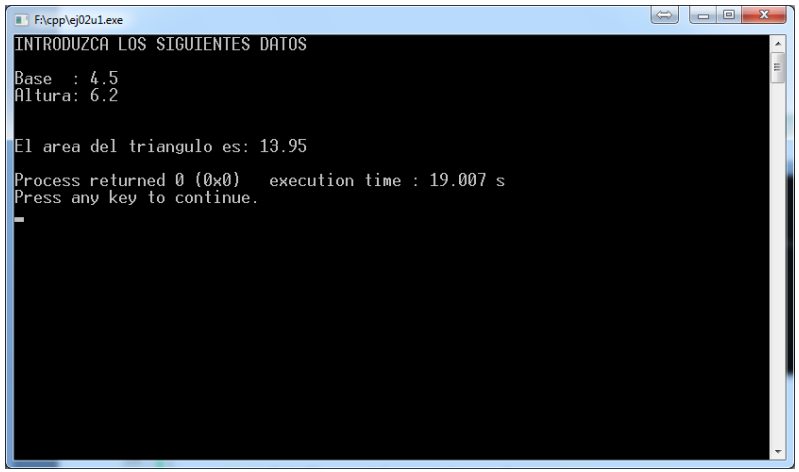
int main(int argc, char** argv)
{
    double base;
    double altura;
    double areaTriangulo;

    cout << "INTRODUZCA LOS SIGUIENTES DATOS"
         << endl << endl;
    cout << "Base   : ";
    cin >> base;
    cout << "Altura: ";
    cin >> altura;

    areaTriangulo = (base * altura) / 2;

    cout << "\n\nEl area del triangulo es: "
         << areaTriangulo
         << endl;

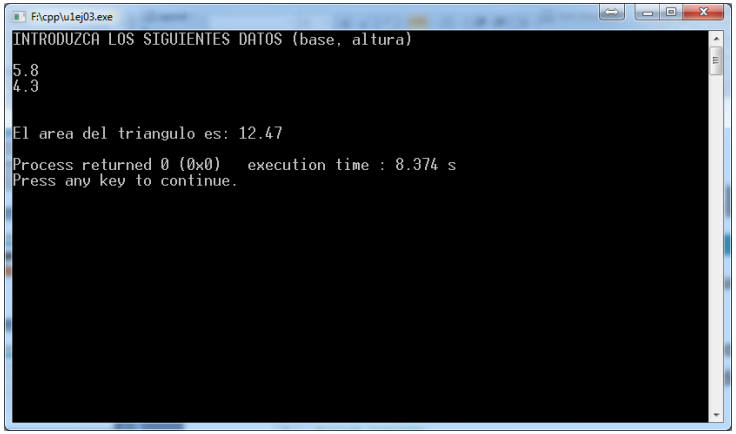
    return 0;
}
```



Lectura de varias variables

- C++ permite combinar en un solo **`cin`** la captura simultánea de varias variables
- Aunque esto no permite que nuestro código sea muy interactivo ya que no es posible insertar un mensaje para indicar que valor ha de pedir el programa, veamos un ejemplo:

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       double base;
8       double altura;
9       double areaTriangulo;
10
11      cout << "INTRODUZCA LOS SIGUIENTES DATOS (base, altura)"
12           << endl << endl;
13
14      cin >> base >> altura;
15
16      areaTriangulo = (base * altura) / 2;
17
18      cout << "\n\nEl area del triangulo es: "
19           << areaTriangulo
20           << endl;
21
22      return 0;
23  }
```



Uso de cin.getline

- **`getline`** nos permite capturar un flujo de entrada estándar, incluyendo los caracteres de espacio en blanco

  ```
  cin.getline(chr, size);
  ```

  Donde:

  `chr` es una variable tipo arreglo de char

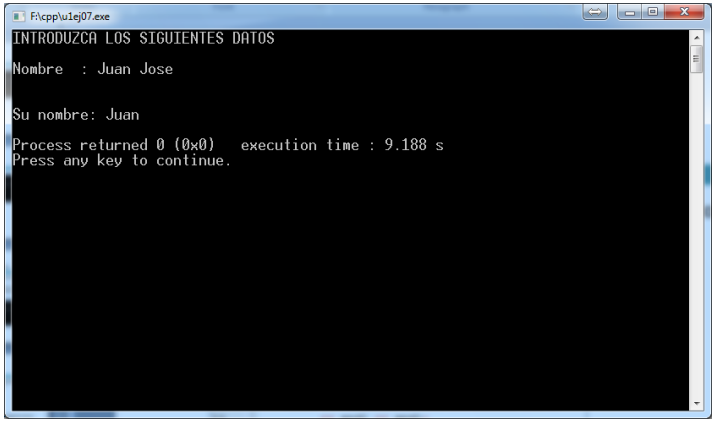
  ejemplo: `char nombre[15];`

  `size` es una variable o literal que especifica el tamano de ***chr***; ***cin*** leerá caracteres hasta la cantidad expresada en ***size***

Uso de cin.getline

- Veamos que pasa si utilizamos ***cin*** para leer una variable ***char*** que incluya caracteres de espacio en blanco:

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       char nombre[20];
8
9       cout << "INTRODUZCA LOS SIGUIENTES DATOS"
10           << endl << endl;
11      cout << "Nombre   : ";
12      cin >> nombre;
13
14      cout << "\n\nSu nombre: " << nombre << endl;
15
16      return 0;
17  }
```

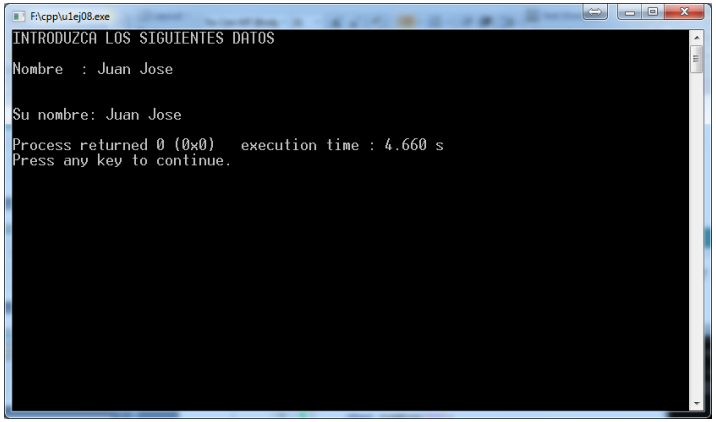


Solamente fue almacenado *Juan*, ya que *cin* encontró un caracter de espacio en blanco

Uso de cin.getline

- Veamos la corrección para el ejemplo anterior mediante el uso de cin.getline:

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       const int TAMANO = 20;
8       char nombre[TAMANO];
9
10      cout << "INTRODUZCA LOS SIGUIENTES DATOS"
11           << endl << endl;
12      cout << "Nombre  : ";
13      cin.getline(nombre, TAMANO);
14
15      cout << "\n\nSu nombre: " << nombre << endl;
16
17      return 0;
18  }
```

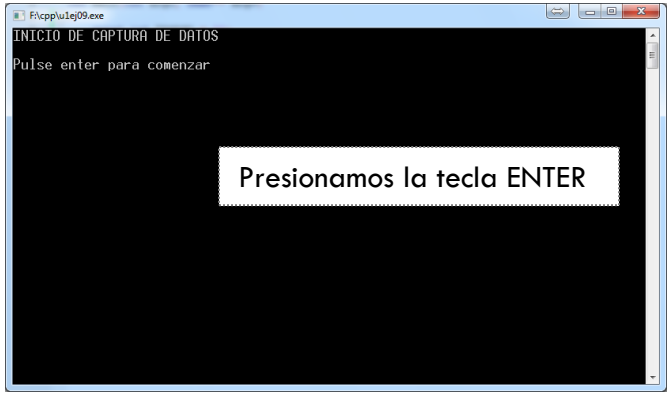


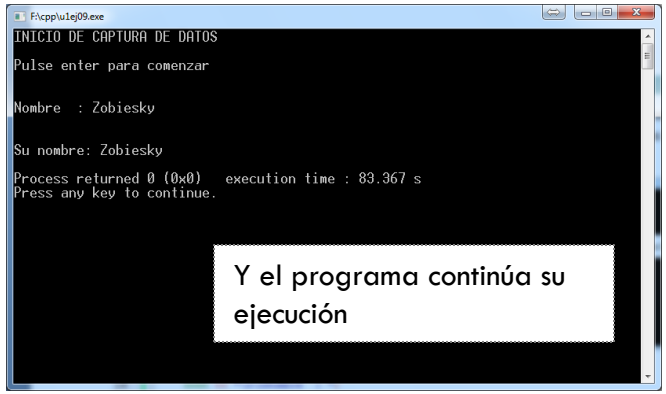
cin.getline permitió leer hasta 20 caracteres incluyendo aquellos tipo espacio en blanco (barra espaciadora, tab, enter)

Uso de cin.get

- Una debilidad del operador >> con las variables tipo ***char*** es que necesitan que sea provisto un caracter, ignorando aquellos caracteres de espacio en blanco, veamos el uso de cin.get para producir una pausa o leer un único caracter (permite enter):

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       const int TAMANO = 20;
8       char nombre[TAMANO];
9       char pausa;
10
11      cout << "INICIO DE CAPTURA DE DATOS"
12           << endl << endl
13           << "Pulse enter para comenzar";
14      cin.get(pausa);
15
16      cout << "\n\nNombre  : ";
17      cin.getline(nombre, TAMANO);
18
19      cout << "\n\nSu nombre: " << nombre << endl;
20
21      return 0;
22  }
```



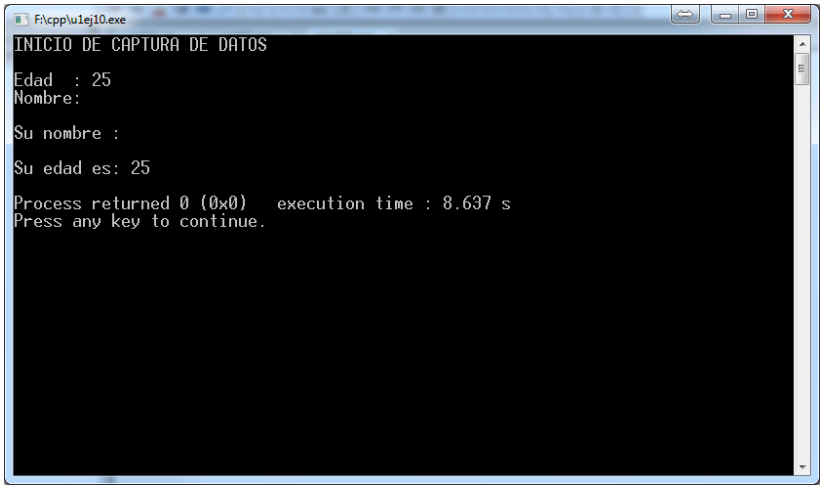


Combinando cin >> y cin.getline / cin.get

- Debemos tener cuidado al combinar `cin >>` con `cin.getline` o con `cin.get`. Si utilizamos primero `cin`, `cin.getline` o `cin.get` no leerá ningún valor, esto es por que cuando `cin` detiene la lectura por que ya encontró el caracter de newline, este caracter permanece en el buffer del teclado y le proporciona a cin.getline y a cin.get un valor para procesar, en resumen, el programa no se dentendrá a esperar que introduzcamos un valor, veamos un ejemplo para ilustrar mejor este caso

Combinando cin >> y cin.getline / cin.get

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       const int TAMANO = 20;
8       char nombre[TAMANO];
9       int edad;
10
11      cout << "INICIO DE CAPTURA DE DATOS"
12           << endl << endl;
13
14      cout << "Edad  : ";
15      cin >> edad;
16
17      cout << "Nombre: ";
18      cin.getline(nombre, TAMANO);
19
20      cout << "\n\nSu nombre : " << nombre << endl
21           << "\nSu edad es: " << edad << endl;
22
23      return 0;
24  }
```



El programa no permitió que se introdujera algún valor para nombre

Uso de cin.ignore

- Mediante el uso de `cin.ignore` podemos solucionar el problema anterior, este recurso nos permite ignorar los caracteres dejados en el buffer del teclado por una sentencia `cin`

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main(int argc, char** argv)
6  {
7      const int TAMANO = 20;
8      char nombre[TAMANO];
9      int edad;
10
11     cout << "INICIO DE CAPTURA DE DATOS"
12          << endl << endl;
13
14     cout << "Edad  : ";
15     cin >> edad;
16     cin.ignore(); // ver aqui uso de cin.ignore
17     cout << "Nombre: ";
18     cin.getline(nombre, TAMANO);
19
20     cout << "\n\nSu nombre : " << nombre << endl
21          << "\nSu edad es: " << edad << endl;
22
23     return 0;
24 }
```



Uso de getline

- **getline** nos permite capturar un flujo de entrada estándar, incluyendo los caracteres de espacio en blanco; `getline` lo utilizamos exclusivamente con variables tipo `string`, no confundir con `cin.getline`

```
getline(cin, str, [delimitador]);
```

Donde:

cin es el recurso para leer desde el flujo estándar de entrada (el teclado)

str es una variable tipo string

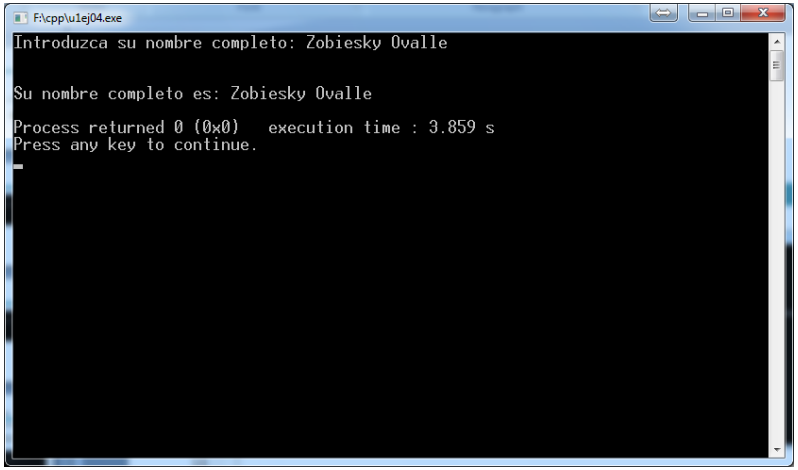
delimitador es un caracter ASCII cualquiera usado para indicarle a ***cin*** hasta donde debe leer el flujo de entrada estándar; si se omite el delimitador por defecto es el caracter de nueva línea o *newline* ('\n'); el delimitador no es almacenado en la variable

Uso de getline

- Las variables tipo ***string*** nos permiten manipular cadenas de caracteres de una forma más cómoda además de proveer un conjunto de recursos asociados a este tipo de datos que le dan una gran versatilidad (más adelante veremos en detalle los ***strings***)
- Lo explicado con *cin.ignore* aplica también para ***getline*** en los ***strings***

Ejemplo con getline

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(int argc, char** argv)
6   {
7       string nombre;
8
9       cout << "Introduzca su nombre completo: ";
10
11      getline(cin, nombre);
12
13      cout << "\n\nSu nombre completo es: "
14           << nombre << endl;
15
16      return 0;
17  }
```



Resumen

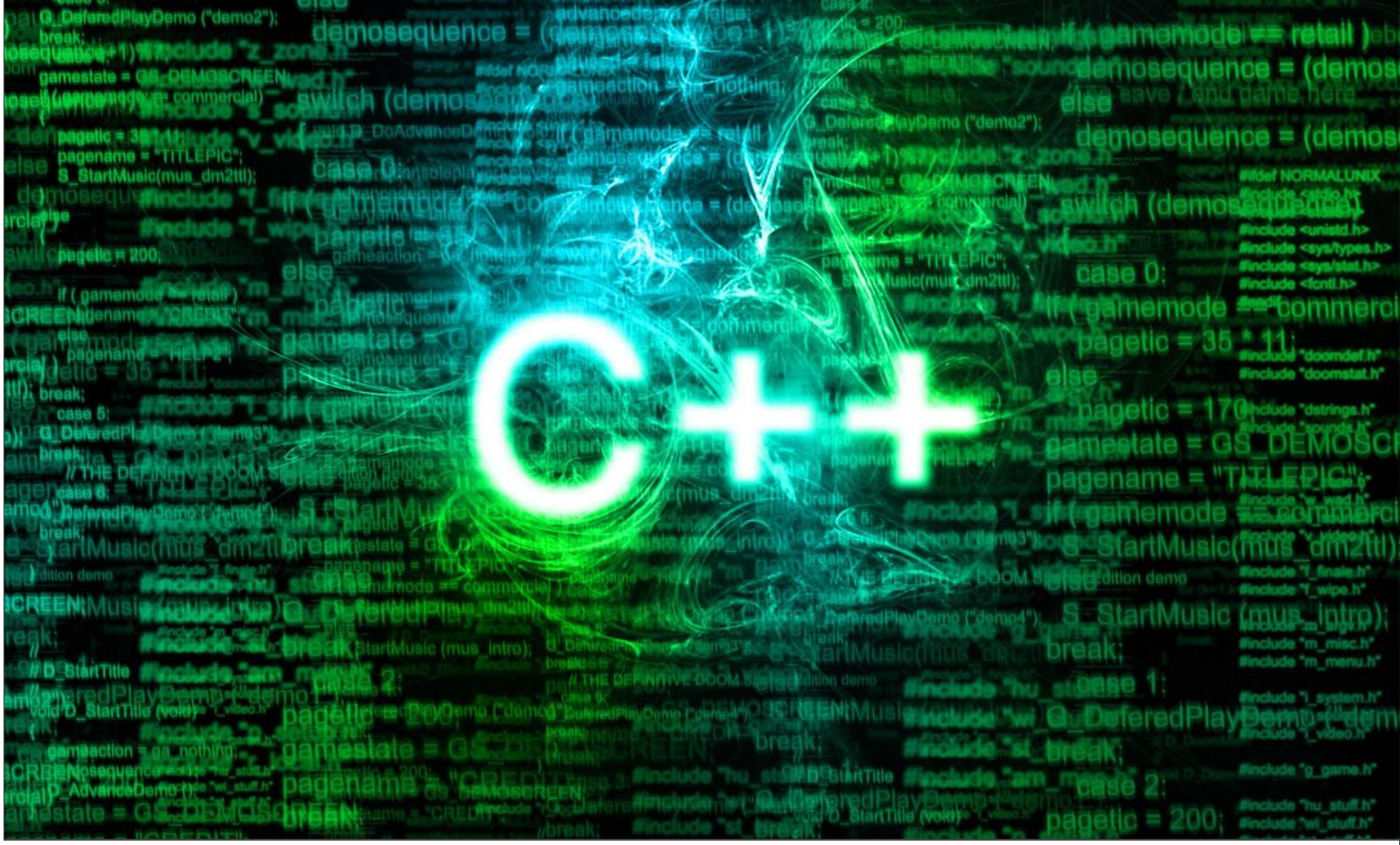
- En esta primera unidad hemos aprendido:
  - Los elementos básicos del lenguaje, como los literales, palabras reservadas, comentarios, sentencias, bloques de código
  - Aprendimos sobre las variables y su alcance dentro de un programa, tipos de datos y rango de valores, el tipo de dato char para manipular caracteres
  - Los principales operadores, de asignación, aritméticos, de relación y lógicos así como sobre la precedencia de operadores y su asociatividad en cuanto a su evaluación, lo que sucede al operar tipos de datos distintos y trabajamos con la conversión explícita de datos o type casting
  - Fue explicado el entorno de trabajo que utilizaremos en el curso y aprendimos que existen otros compiladores y editores de C++, nos familiarizamos con terminos como IDE y SDK. Vimos los pasos para la compilación de un programa

Resumen

- En esta primera unidad hemos aprendido:
  - No existe un libro o manual mágico para construir algoritmos pero sí es necesario conocer como resolver el problema de forma no automatizada o manual. La estrategia consiste en un enfoque de Entrada-Proceso-Salida para entender mejor como debemos resolver o diseñar los algoritmos. Vimos los pasos para construir una aplicación C++
  - Escribimos nuestro primer programa C++ y conocimos en mas detalle sobre el alcance de las variables asi como tambien el concepto de nombre de espacio
  - Finalmente vimos ampliamente las operaciones de entrada y salida (cin, cout), ejemplos con programas, el uso de cin.getline, cin.get para variables tipo char y getline para variables tipo string y que sucede al combinar cin con getline o cin.getline

Bibliografía

- Addison Wesley, Tony Gaddis. (2009). Starting Out with C++ Brief: From Control Structures through Objects. San Francisco, California. 6th Edition.
- Aprende a programar en C++. (2010). Página dedicada a la enseñanza de C++ con libro de libre descarga, recuperado el 3 Junio 2013 de: http://c.conclase.net/curso/index.php
- Página dedicada al lenguaje de programación C++ (2000 – 2014). Tutoriales y recursos de libre descarga, recuperado Enero 2014 en: http://www.cplusplus.com All rights reserved - v3.1
- Paul Deitel, Harvey Deitel (2011). C++ How to Program, New York, 8th Edition, Prentice Hall.



Unidad 1: Iniciando con C++

Por Félix Ferreiras y Zobiesky Ovalle